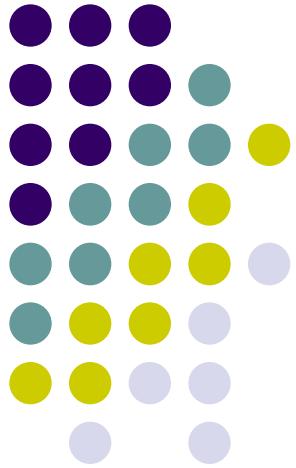
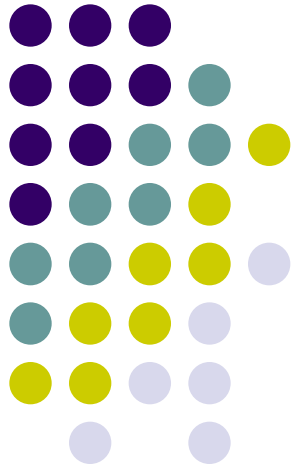
Chapter 1. Basic Structure of Computers



Functional Units



Functional Units

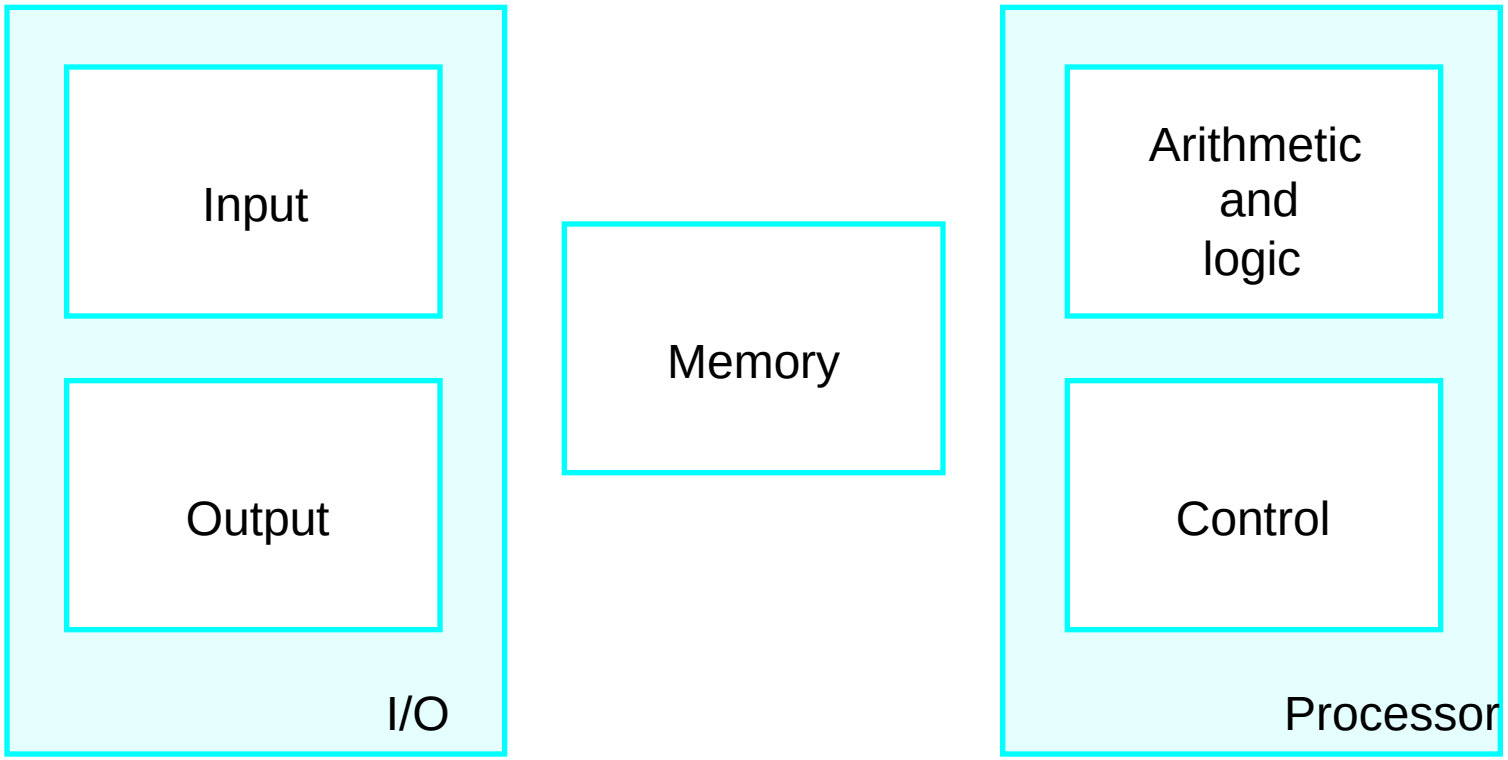
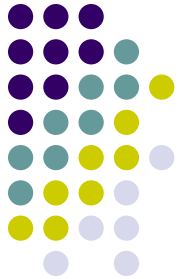


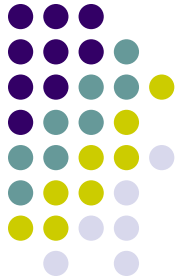
Figure 1.1. Basic functional units of a computer.

Information Handled by a Computer



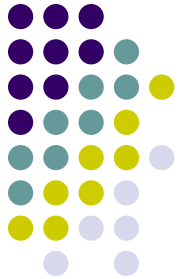
- Instructions/machine instructions
  - Govern the transfer of information within a computer as well as between the computer and its I/O devices
  - Specify the arithmetic and logic operations to be performed
  - Program
- Data
  - Used as operands by the instructions
  - Source program
- Encoded in binary code – 0 and 1

Memory Unit



- Store programs and data
- Two classes of storage
  - Primary storage
    - Fast
    - Programs must be stored in memory while they are being executed
    - Large number of semiconductor storage cells
    - Processed in words
    - Address
    - RAM and memory access time
    - Memory hierarchy – cache, main memory
  - Secondary storage – larger and cheaper

Arithmetic and Logic Unit (ALU)

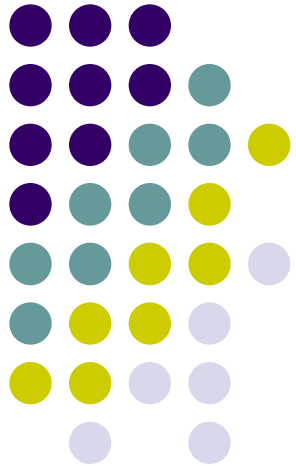


- Most computer operations are executed in ALU of the processor.
- Load the operands into memory – bring them to the processor – perform operation in ALU – store the result back to memory or retain in the processor.
- Registers
- Fast control of ALU

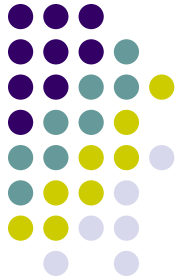
Control Unit

- All computer operations are controlled by the control unit.
- The timing signals that govern the I/O transfers are also generated by the control unit.
- Control unit is usually distributed throughout the machine instead of standing alone.
- Operations of a computer:
  - Accept information in the form of programs and data through an input unit and store it in the memory
  - Fetch the information stored in the memory, under program control, into an ALU, where the information is processed
  - Output the processed information through an output unit
  - Control all activities inside the machine through a control unit

Basic Operational Concepts

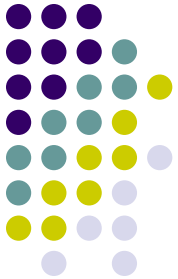


Review



- Activity in a computer is governed by instructions.
- To perform a task, an appropriate program consisting of a list of instructions is stored in the memory.
- Individual instructions are brought from the memory into the processor, which executes the specified operations.
- Data to be used as operands are also stored in the memory.

Separate Memory Access and ALU Operation



- Load R2, LOC ; R2 ← [LOC]
- Add R4, R2, R3 ; R4 ← [R2] +[R3]
- Store R4, LOC ; [LOC] ← R4
- Whose contents will be overwritten?

Connection Between the Processor and the Memory

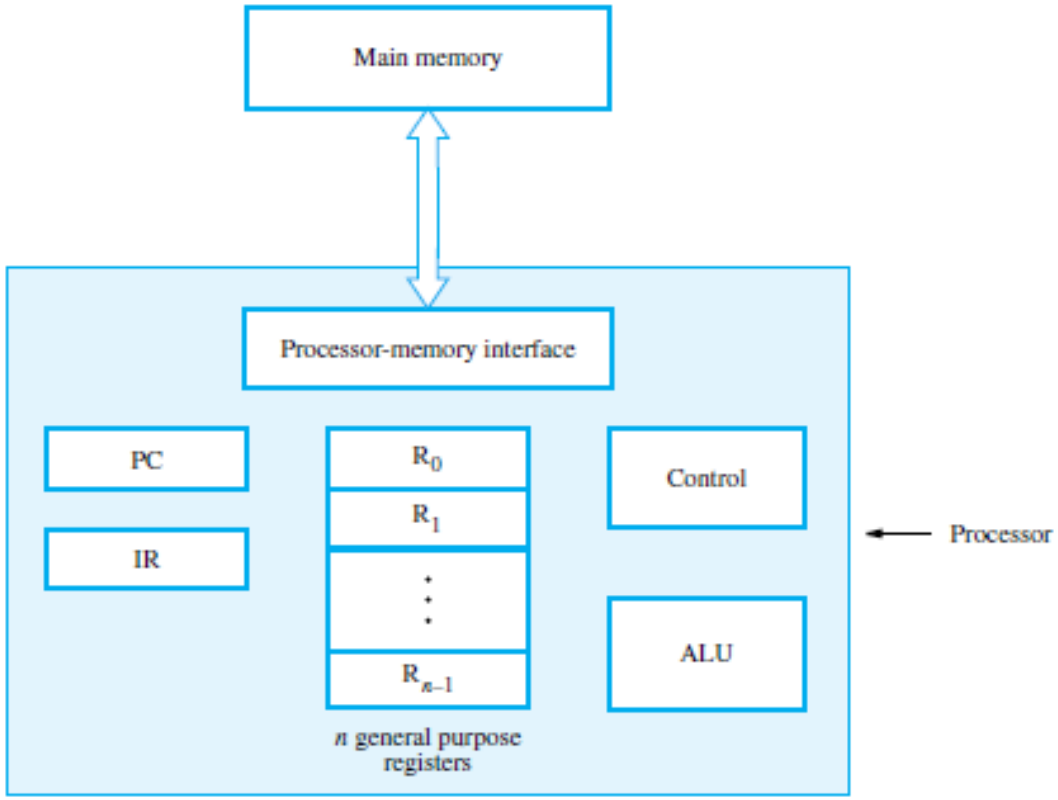
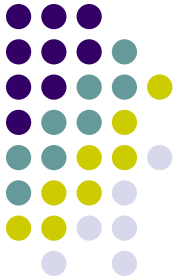


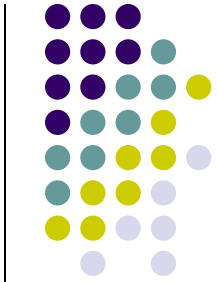
Figure 1.2 Connection between the processor and the main memory.

Registers



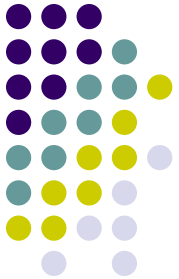
- Instruction register (IR)
- Program counter (PC)
- General-purpose register (R_0 – R_{n-1})
- Processor Memory Interface

Typical Operating Steps



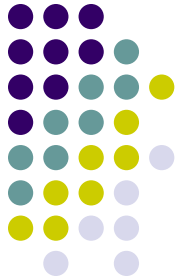
- Programs reside in the memory through input devices
- PC is set to point to the first instruction
- The contents of PC are transferred to Processor mem. interface
- A Read signal is sent to the memory
- The first instruction is read out and loaded into Processor mem. interface and trasferred to IR
- Decode and execute the instruction

Typical Operating Steps (Cont')

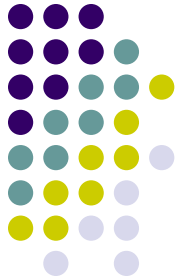


- Get operands for ALU
  - General-purpose register
  - Memory (Address from Processor memory interface – Read signal)
- Perform operation in ALU
- Store the result back
  - To general-purpose register
  - To memory (Address and data from Processor memory interface – Write signal)
- During the execution, PC is incremented to the next instruction

Interrupt

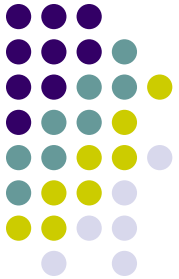


- Normal execution of programs may be preempted if some device requires urgent servicing.
- The normal execution of the current program must be interrupted – the device raises an *interrupt* signal.
- Interrupt-service routine
- Current system information backup and restore (PC, general-purpose registers, control information, specific information)



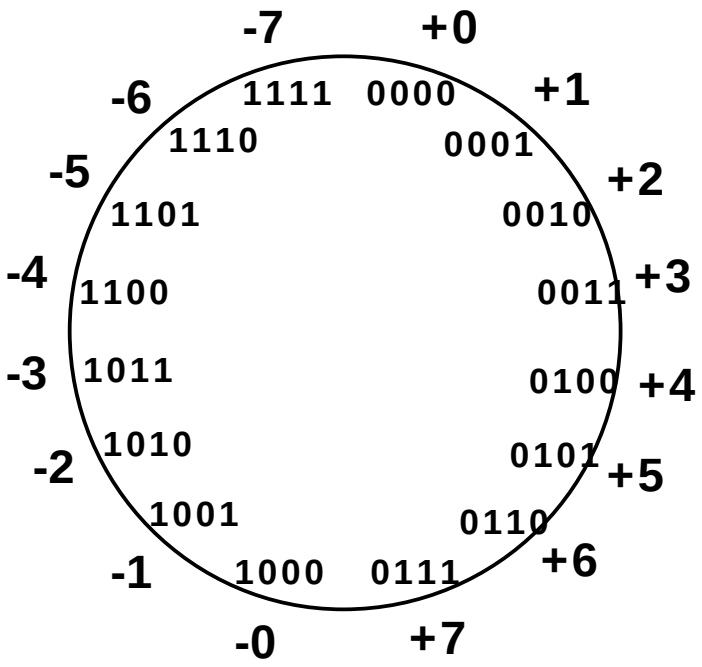
Number Representation and Arithmetic Operations

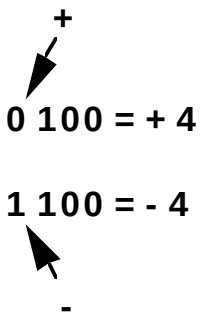
Signed Integer



- 3 major representations:
  - Sign and magnitude
  - One’s complement
  - Two’s complement
- Assumptions:
  - 4-bit machine word
  - 16 different values can be represented
  - Roughly half are positive, half are negative

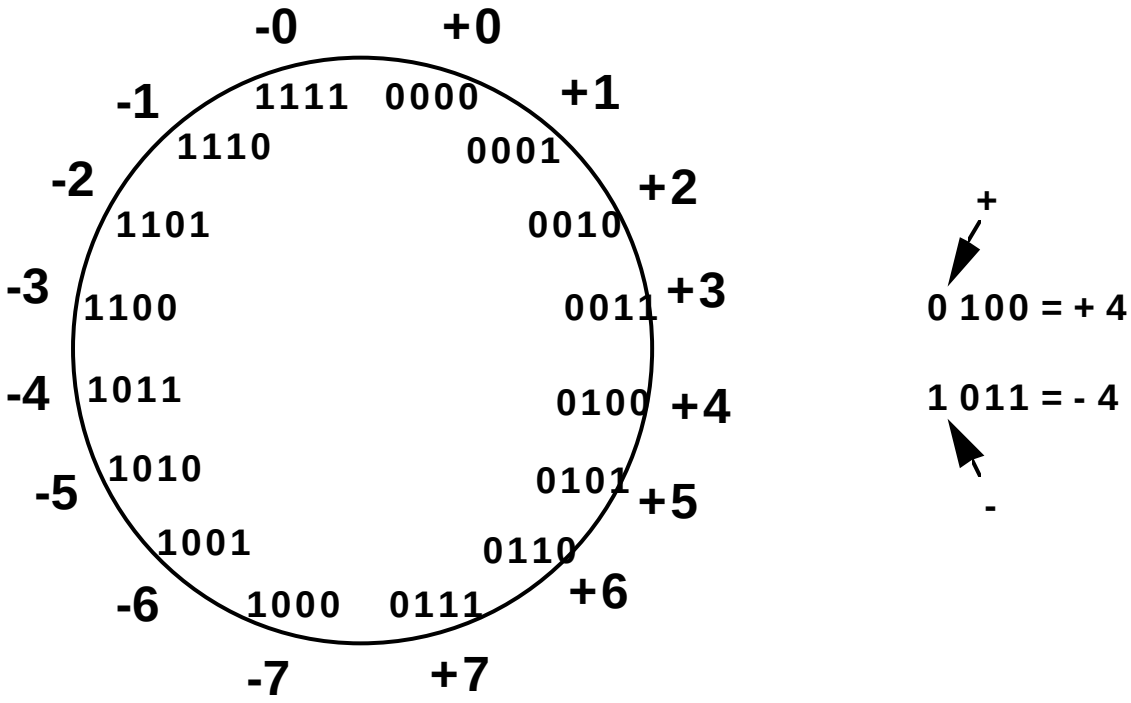
Sign and Magnitude Representation (Mod-16 system)





High order bit is sign: 0 = positive (or zero), 1 = negative
Three low order bits is the magnitude: 0 (000) thru 7 (111)
Number range for n bits = +/-2^{n-1} -1
Two representations for 0

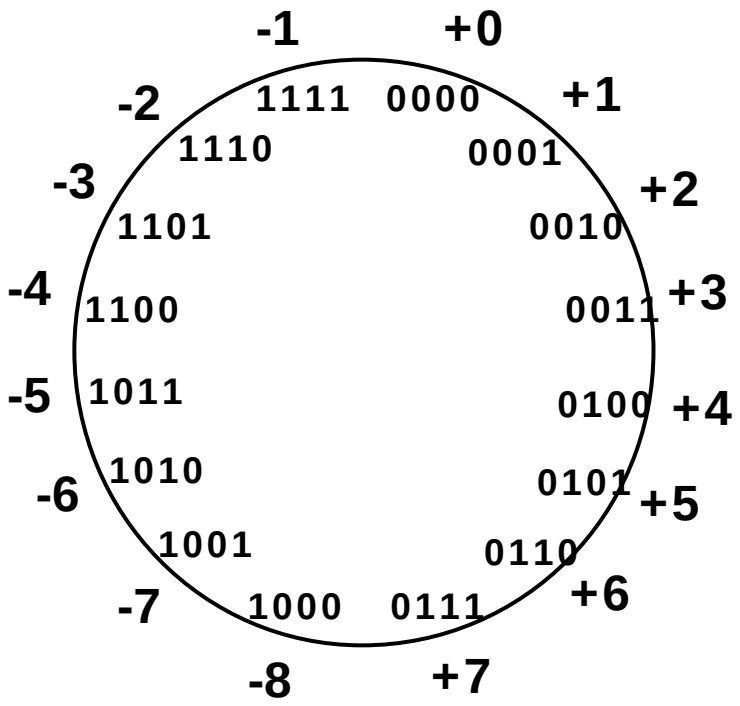
One's Complement Representation

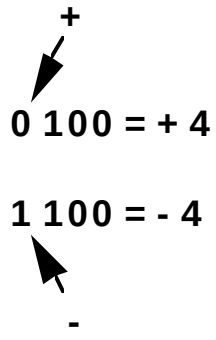


- Subtraction implemented by addition & 1's complement
- Still two representations of 0! This causes some problems
- Some complexities in addition

Two's Complement Representation

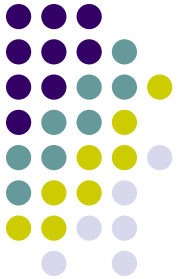
like 1's comp except shifted one position clockwise





- Only one representation for 0
- One more negative number than positive number

Binary, Signed-Integer Representations

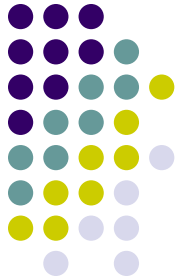


Page 28

B	Values represented		
$b_3 b_2 b_1 b_0$	Sign and magnitude	1's complement	2's complement
0 1 1 1	+ 7	+ 7	+ 7
0 1 1 0	+ 6	+ 6	+ 6
0 1 0 1	+ 5	+ 5	+ 5
0 1 0 0	+ 4	+ 4	+ 4
0 0 1 1	+ 3	+ 3	+ 3
0 0 1 0	+ 2	+ 2	+ 2
0 0 0 1	+ 1	+ 1	+ 1
0 0 0 0	+ 0	+ 0	+ 0
1 0 0 0	- 0	- 7	- 8
1 0 0 1	- 1	- 6	- 7
1 0 1 0	- 2	- 5	- 6
1 0 1 1	- 3	- 4	- 5
1 1 0 0	- 4	- 3	- 4
1 1 0 1	- 5	- 2	- 3
1 1 1 0	- 6	- 1	- 2
1 1 1 1	- 7	- 0	- 1

Figure 2.1. Binary, signed-integer representations.

Addition and Subtraction – 2's Complement



If carry-in to the high order bit = carry-out then ignore carry

4	0100	-4	1100
+ 3	0011	+ (-3)	1101
7	0111	-7	11001

if carry-in differs from carry-out then overflow

4	0100	-4	1100
- 3	1101	+ 3	0011
1	10001	-1	1111

Simpler addition scheme makes twos complement the most common choice for integer number systems within digital systems

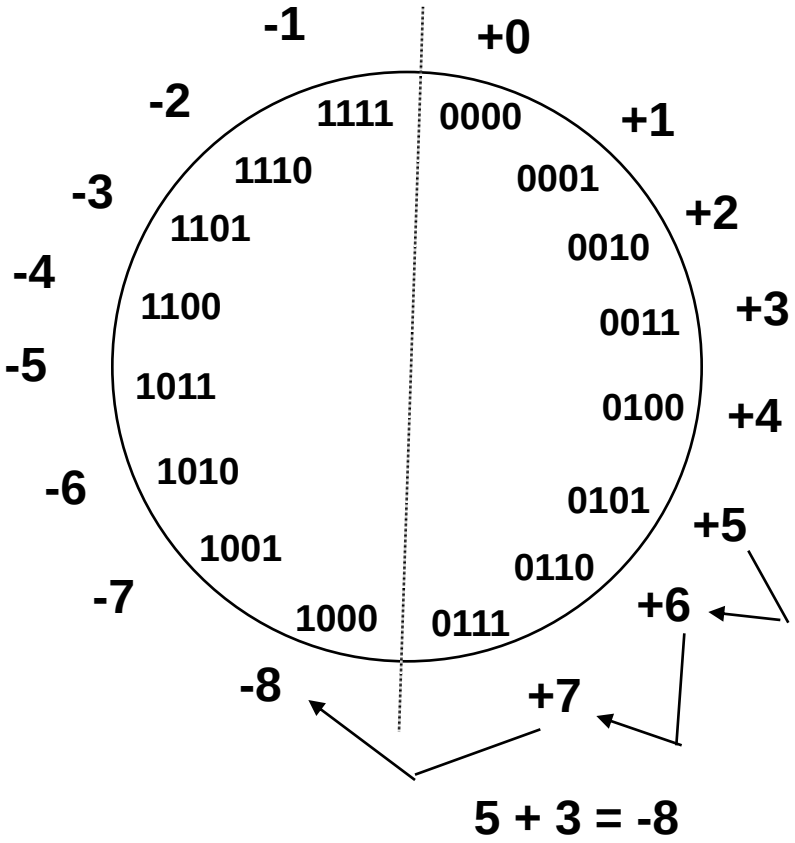
2’s-Complement Add and Subtract Operations

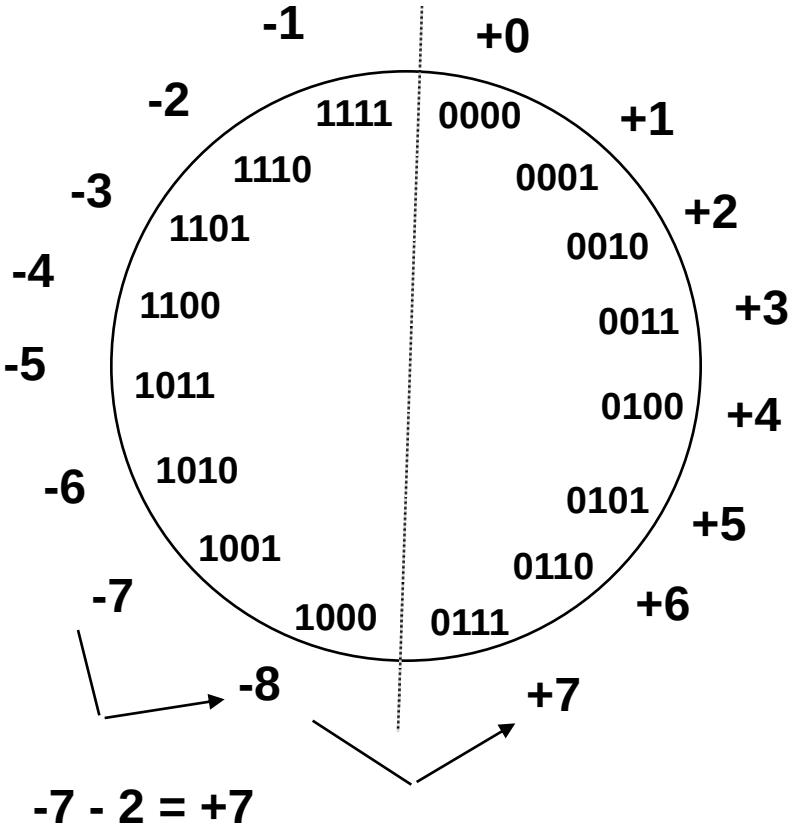
Page 31

(a)	0 0 1 0	(+2)		b)	0 1 0 0	(+4)
	+ 0 0 1 1	(+3)			+ 1 0 1 0	(- 6)
	0 1 0 1	(+5)			1 1 1 0	(- 2)
(c)	1 0 1 1	(- 5)		(d)	0 1 1 1	(+7)
	+ 1 1 1 0	(- 2)			+ 1 1 0 1	(- 3)
	1 0 0 1	(- 7)			0 1 0 0	(+4)
(e)	1 1 0 1	(- 3)	⇒		1 1 0 1	
	- 1 0 0 1	(- 7)			+ 0 1 1 1	
					0 1 0 0	(+4)
(f)	0 0 1 0	(+2)	⇒		0 0 1 0	
	- 0 1 0 0	(+4)			+ 1 1 0 0	
					1 1 1 0	(- 2)
(g)	0 1 1 0	(+6)	⇒		0 1 1 0	
	- 0 0 1 1	(+3)			+ 1 1 0 1	
					0 0 1 1	(+3)
(h)	1 0 0 1	(- 7)	⇒		1 0 0 1	
	- 1 0 1 1	(- 5)			+ 0 1 0 1	
					1 1 1 0	(- 2)
(i)	1 0 0 1	(- 7)	⇒		1 0 0 1	
	- 0 0 0 1	(+1)			+ 1 1 1 1	
					1 0 0 0	(- 8)
(j)	0 0 1 0	(+2)	⇒		0 0 1 0	
	- 1 1 0 1	(- 3)			+ 0 0 1 1	
					0 1 0 1	(+5)

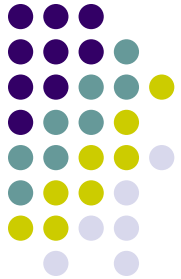
Figure 2.4. 2's-complement Add and Subtract operations.

Overflow - Add two positive numbers to get a negative number or two negative numbers to get a positive number





Overflow Conditions



```
      0 1 1 1
 5    0 1 0 1
 3    0 0 1 1
-8    1 0 0 0
```

Overflow

```
      1 0 0 0
-7    1 0 0 1
-2    1 1 0 0
 7   1 0 1 1 1
```

Overflow

```
      0 0 0 0
 5    0 1 0 1
 2    0 0 1 0
 7    0 1 1 1
```

No overflow

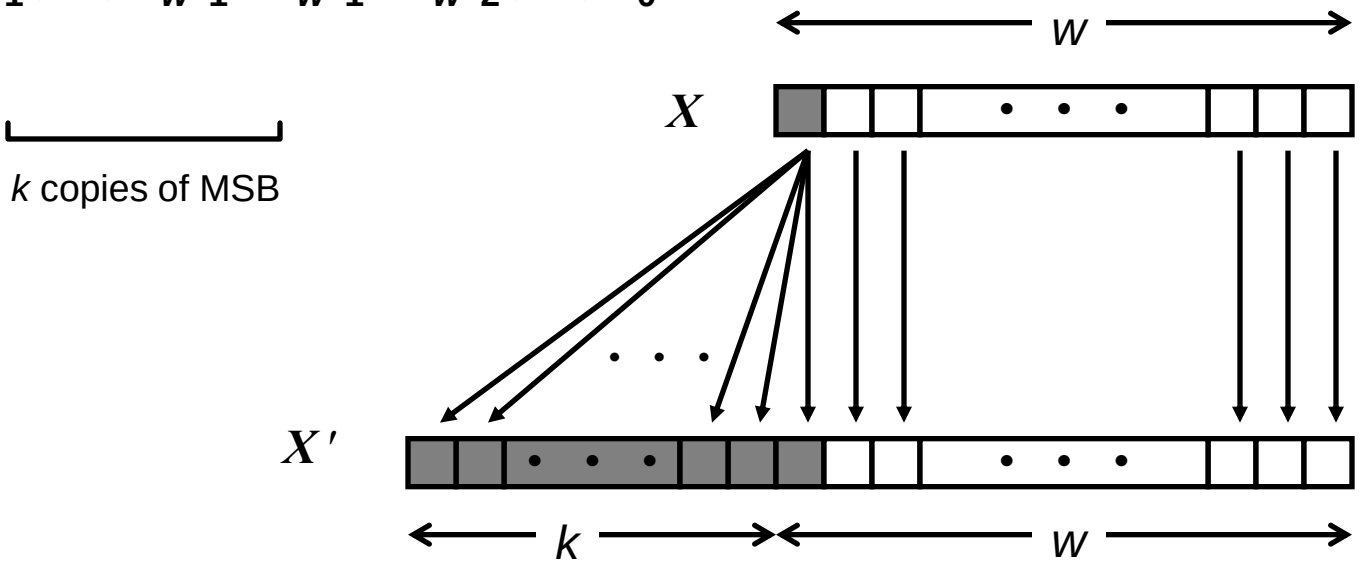
```
      1 1 1 1
-3    1 1 0 1
-5    1 0 1 1
-8   1 1 0 0 0
```

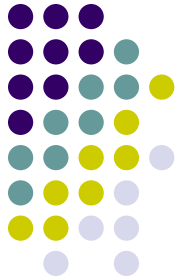
No overflow

Overflow when carry-in to the high-order bit does not equal carry out

Sign Extension

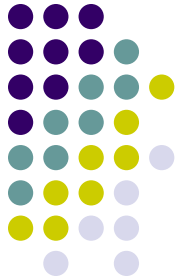
- Task:
  - Given *w*-bit signed integer *x*
  - Convert it to *w*+*k*-bit integer with same value
- Rule:
  - Make *k* copies of sign bit:
  - $X' = x_{w-1}, \ldots, x_{w-1}, x_{w-1}, x_{w-2}, \ldots, x_0$



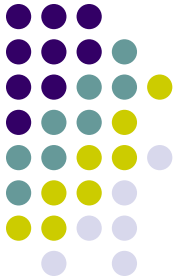


Chapter 2. Instruction Set Architecture

Objectives



- Machine instructions and program execution, including branching and subroutine call and return operations.
- Addressing methods for accessing register and memory operands.
- Assembly language for representing machine instructions, data, and programs.
- Program-controlled Input/Output operations.



Memory Locations and Addresses

Memory Location, Addresses, and Operation

- Memory consists of many millions of storage cells, each of which can store 1 bit.
- Data is usually accessed in *n*-bit groups. *n* is called word length.

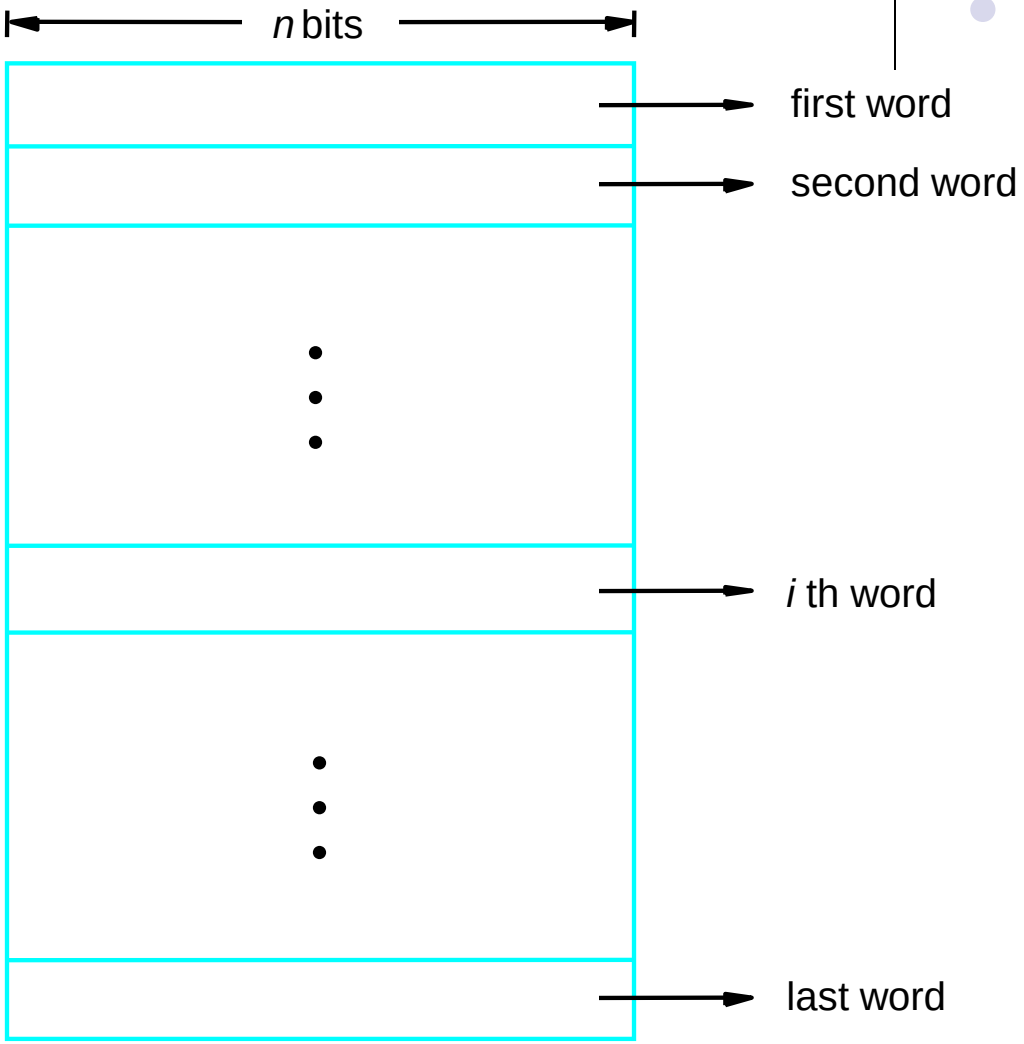
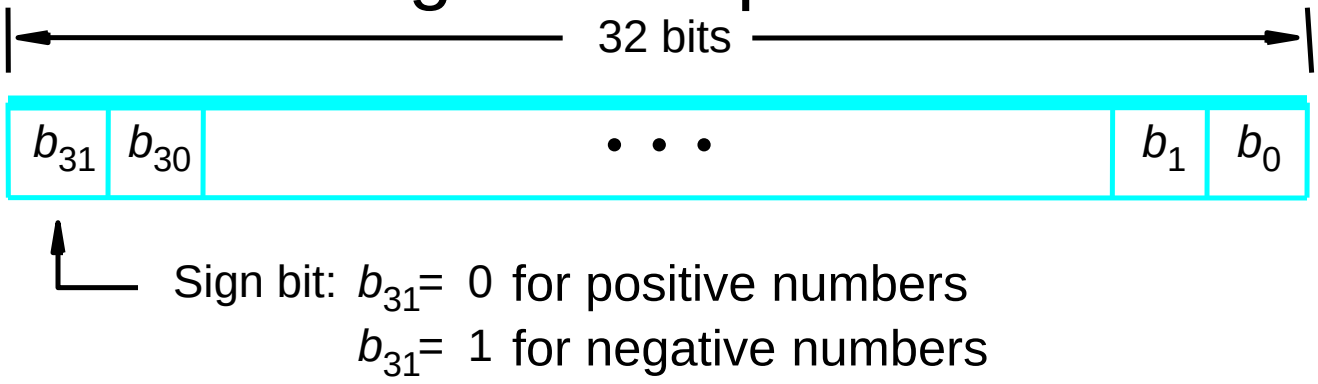


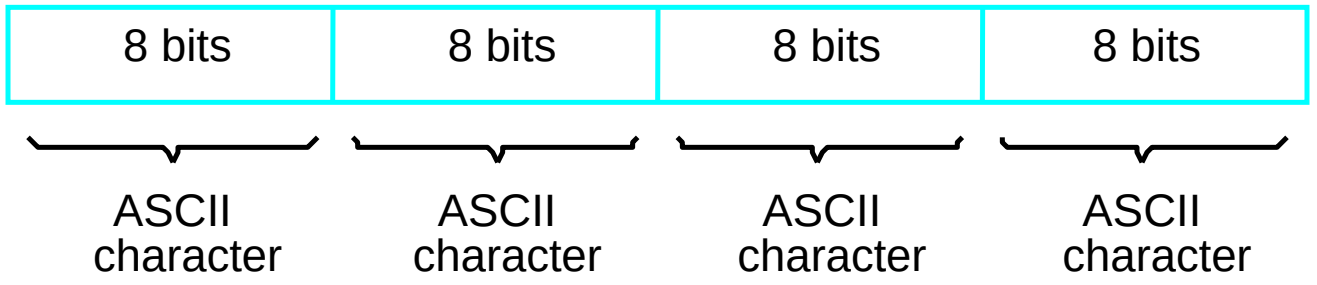
Figure 2.5. Memory words.

Memory Location, Addresses, and Operation

- 32-bit word length example

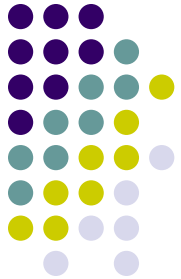


(a) A signed integer



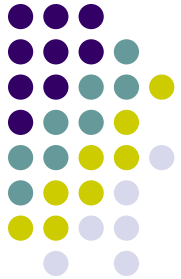
(b) Four characters

Memory Location, Addresses, and Operation



- To retrieve information from memory, either for one word or one byte (8-bit), addresses for each location are needed.
- A *k*-bit address memory has 2^k memory locations, namely 0 – 2^k-1, called memory space.
- 24-bit memory: 2^{24} = 16,777,216 = 16M (1M=2^{20})
- 32-bit memory: 2^{32} = 4G (1G=2^{30})
- 1K(kilo)=2^{10}
- 1T(tera)=2^{40}

Memory Location, Addresses, and Operation



- It is impractical to assign distinct addresses to individual bit locations in the memory.
- The most practical assignment is to have successive addresses refer to successive byte locations in the memory – byte-addressable memory.
- Byte locations have addresses 0, 1, 2, … If word length is 32 bits, they successive words are located at addresses 0, 4, 8,…

Big-Endian and Little-Endian Assignments

Big-Endian: lower byte addresses are used for the most significant bytes of the word

Little-Endian: opposite ordering. lower byte addresses are used for the less significant bytes of the word

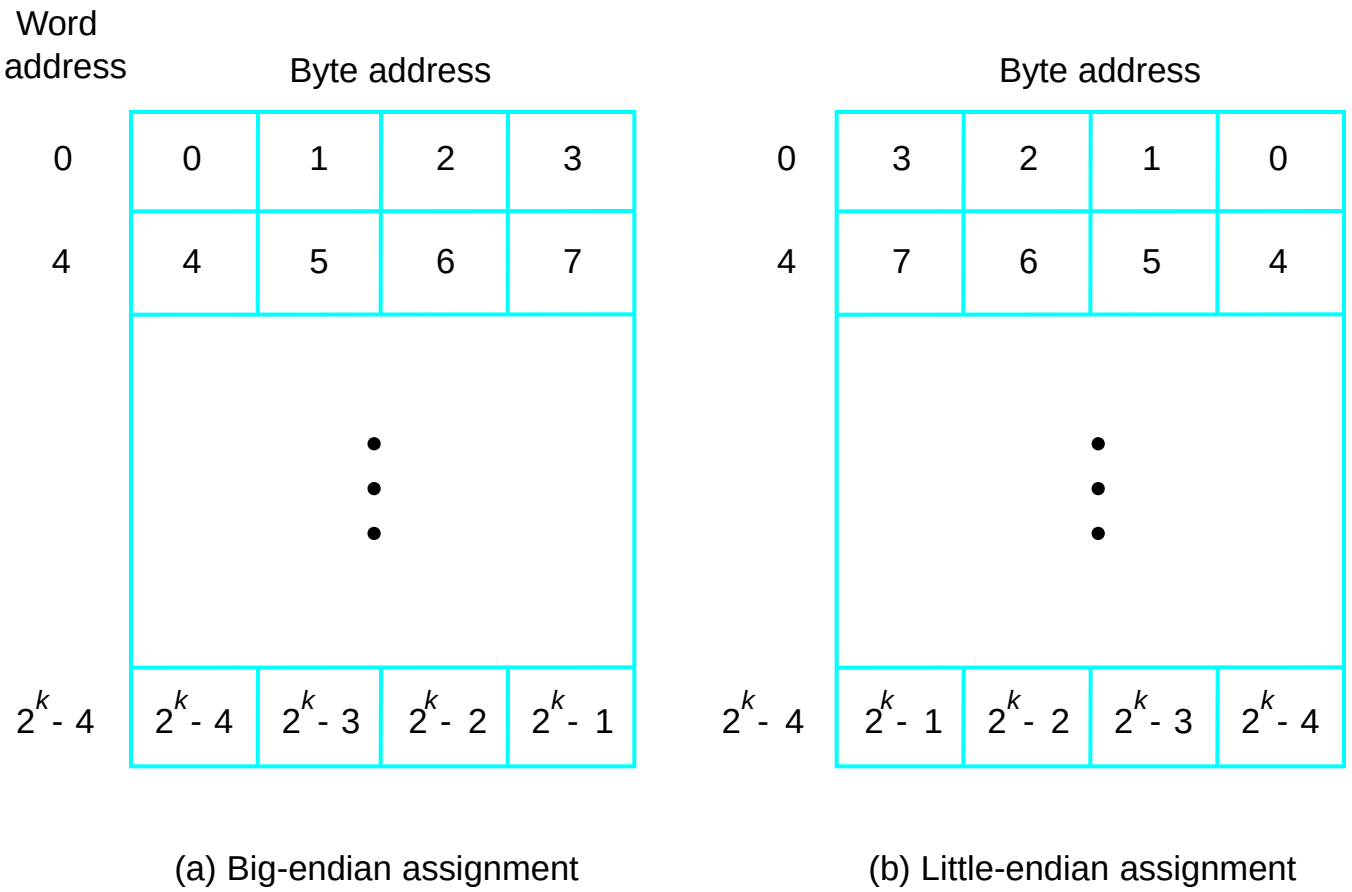
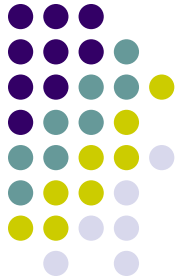


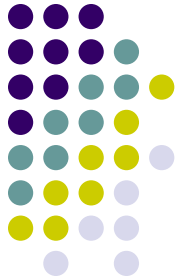
Figure 2.7. Byte and word addressing.

Memory Location, Addresses, and Operation



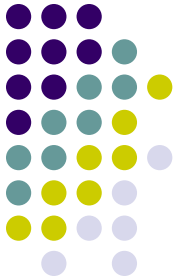
- Address ordering of bytes
- Word alignment
  - Words are said to be aligned in memory if they begin at a byte addr. that is a multiple of the num of bytes in a word.
    - 16-bit word: word addresses: 0, 2, 4,….
    - 32-bit word: word addresses: 0, 4, 8,….
    - 64-bit word: word addresses: 0, 8,16,….
- Access numbers, characters, and character strings

Memory Operation

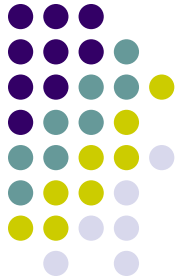


- Load (or Read or Fetch)
  - Copy the content. The memory content doesn’t change.
  - Address – Load
  - Registers can be used
- Store (or Write)
  - Overwrite the content in memory
  - Address and Data – Store
  - Registers can be used

Instruction and Instruction Sequencing

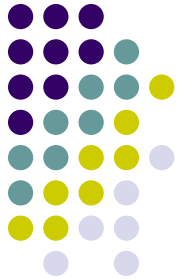


“Must-Perform” Operations



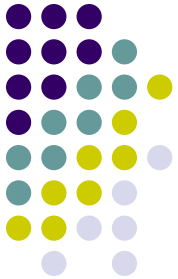
- Data transfers between the memory and the processor registers
- Arithmetic and logic operations on data
- Program sequencing and control
- I/O transfers

Register Transfer Notation



- Identify a location by a symbolic name standing for its hardware binary address (LOC, R0,…)
- Contents of a location are denoted by placing square brackets around the name of the location (R1←[LOC], R3 ←[R1]+[R2])
- Register Transfer Notation (RTN)

Assembly Language Notation



- Represent machine instructions and programs.
- Load R1, LOCA ; R1←[LOCA]
- Store R2, LOCA, ; [LOCA] ← [R2]
- Add R3, R1, R2 ; R3 ←[R1]+[R2]

Instruction Formats

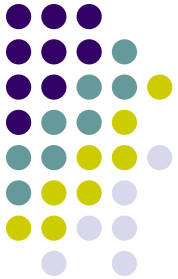
- RISC Instructions
  - Lots of registers. Memory is restricted to Load & Store

    C = A + B; HLL

    Load R2, A ; for RISC machine
    Load R3,B
    Add R4, R2, R3
    Store R4,C

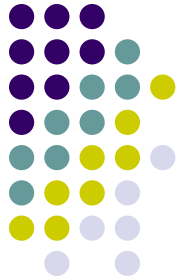


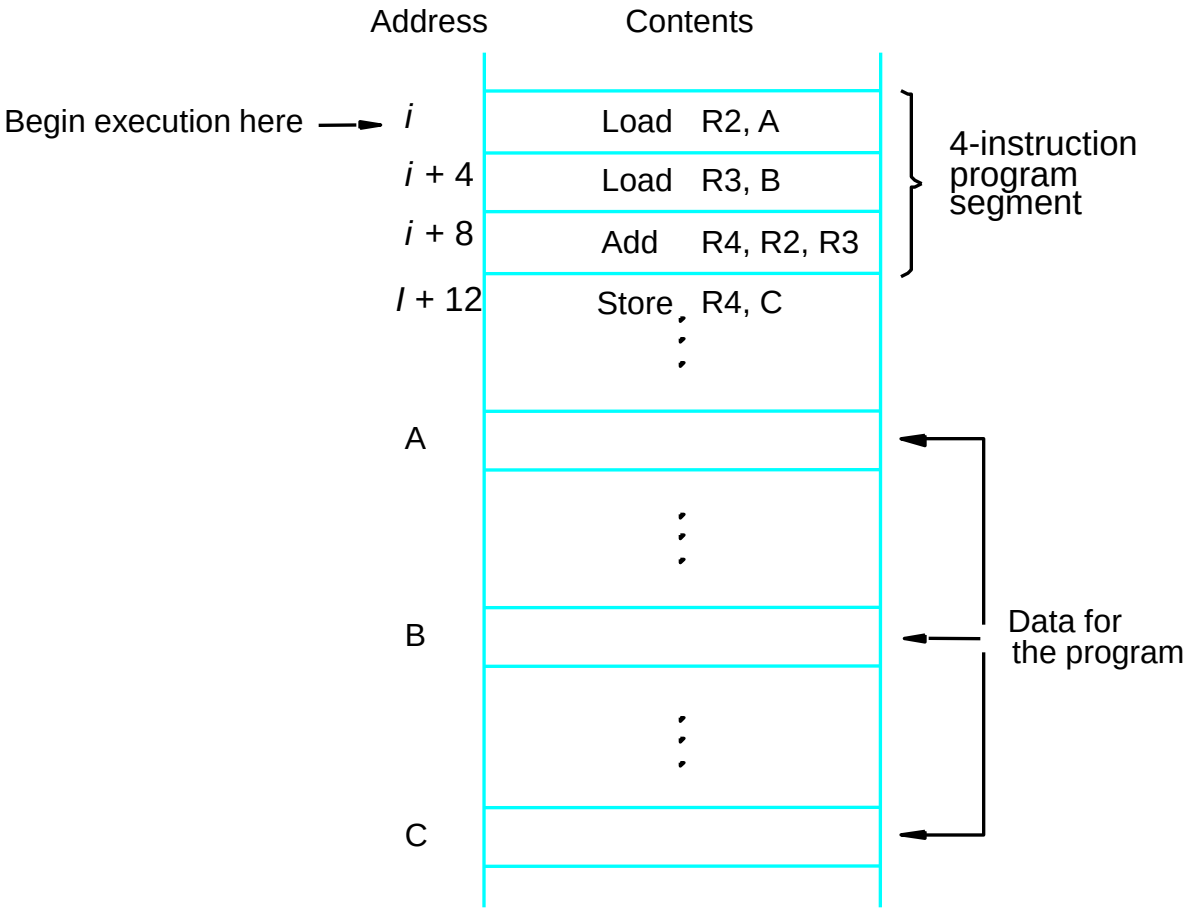
Using Registers



- Registers are faster
- Shorter instructions
  - The number of registers is smaller (e.g. 32 registers need 5 bits)
- Potential speedup
- Minimize the frequency with which data is moved back and forth between the memory and processor registers.

Instruction Execution and Straight-Line Sequencing





Assumptions:
- One memory operand per instruction
- 32-bit word length
- Memory is byte addressable
- Full memory address can be directly specified in a single-word instruction

Two-phase procedure
-Instruction fetch
-Instruction execute

Figure 2.8. A program for C ← [A] + [B].

Branching

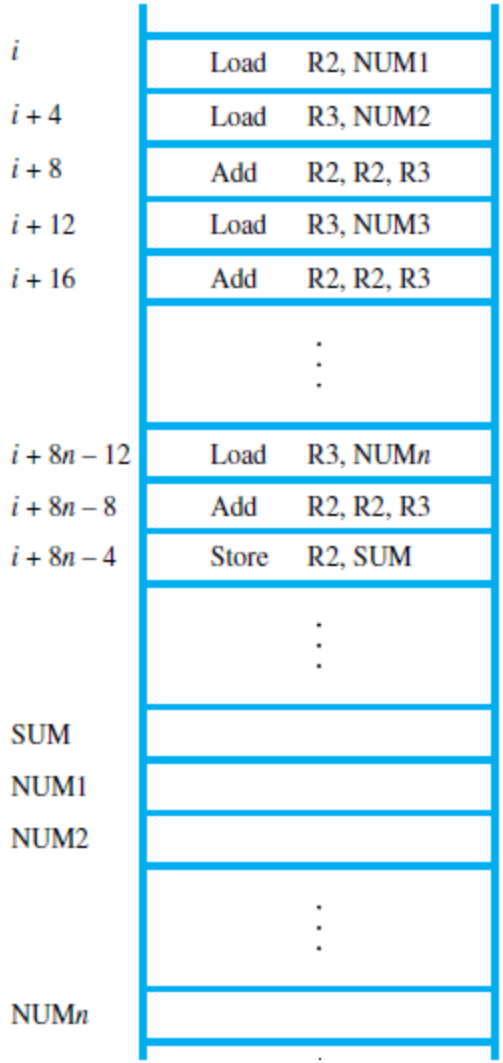


Figure 2.9. A straight-line program for adding *n* numbers.

Branching

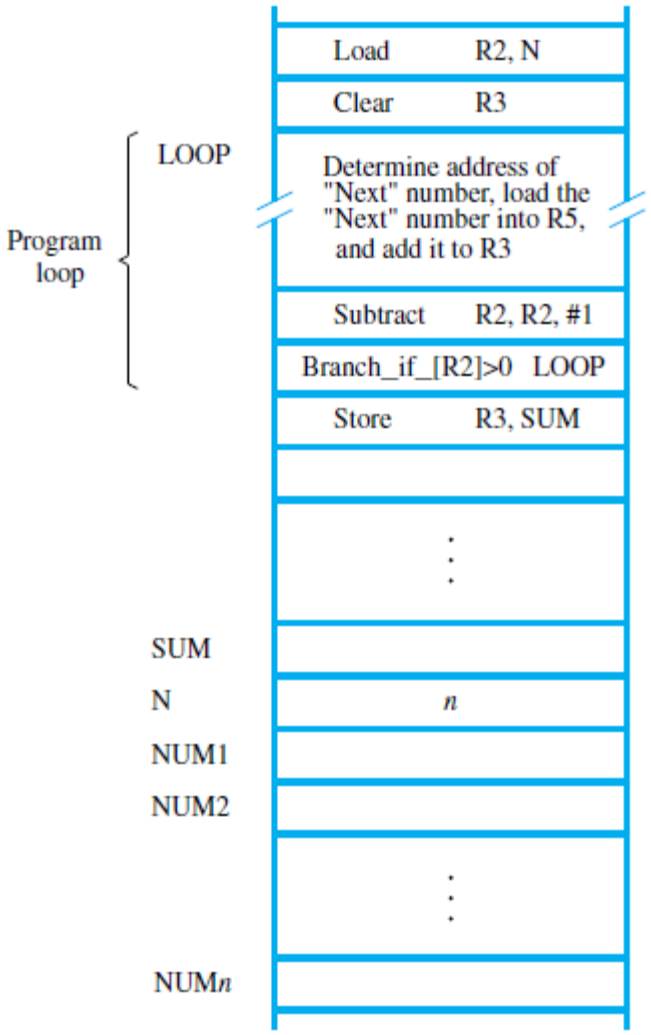
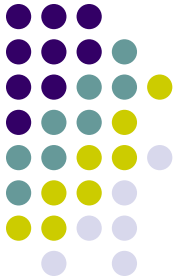


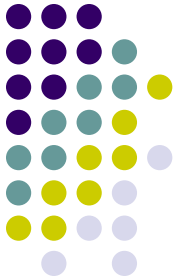
Figure 2.9. Using a loop to add n numbers.

Condition Codes

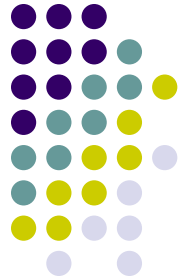


- Condition code flags
- Condition code register / status register
- N (negative)
- Z (zero)
- V (overflow)
- C (carry)
- Different instructions affect different flags

Addressing Modes



Generating Memory Addresses



- How to specify the address of branch target?
- Can we give the memory operand address directly in a single Add instruction in the loop?
- Use a register to hold the address of NUM1; then increment by 4 on each pass through the loop.

Addressing Modes

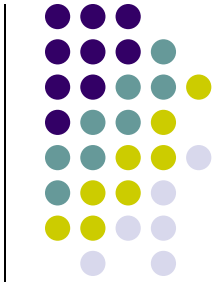


Table 2.1 RISC-type addressing modes.

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	R*i*	EA = R*i*
Absolute	LOC	EA = LOC
Register indirect	(R*i*)	EA = [R*i*]
Index	X(R*i*)	EA = [R*i*] + X
Base with index	(R*i*,R*j*)	EA = [R*i*] + [R*j*]

EA = effective address
Value = a signed number
X = index value

Addressing Modes



- Register
  - The operand is the contents of a processor register; the name of the register is given in the instruction.

    MOV R1, R2
- Absolute mode
  - .The operand is in a memory location; the address of this location is given explicitly in the instruction.

    Load R4, LOCA
- Immediate
  - The operand is given explicitly in the instruction

    Add R4, R6, #200

Addressing Modes

- Indirect Address
  - Indicate the memory location that holds the address of the memory location that holds the data

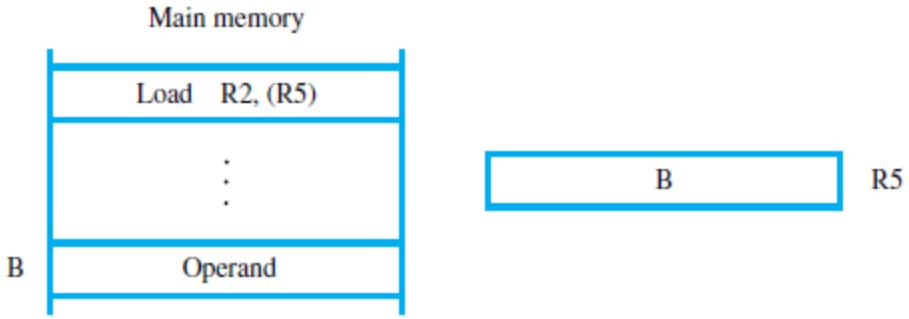
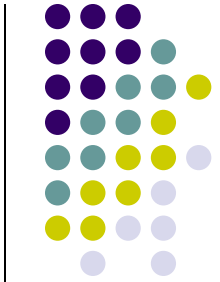


Figure 2.7 Register indirect addressing.

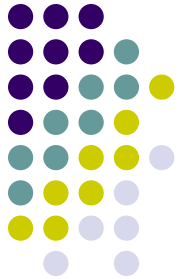
Addressing Modes



	Load	R2, N	Load the size of the list.
	Clear	R3	Initialize sum to 0.
	Move	R4, #NUM1	Get address of the first number.
LOOP:	Load	R5, (R4)	Get the next number.
	Add	R3, R3, R5	Add this number to sum.
	Add	R4, R4, #4	Increment the pointer to the list.
	Subtract	R2, R2, #1	Decrement the counter.
	Branch_if_[R2]>0	LOOP	Branch back if not finished.
	Store	R3, SUM	Store the final sum.

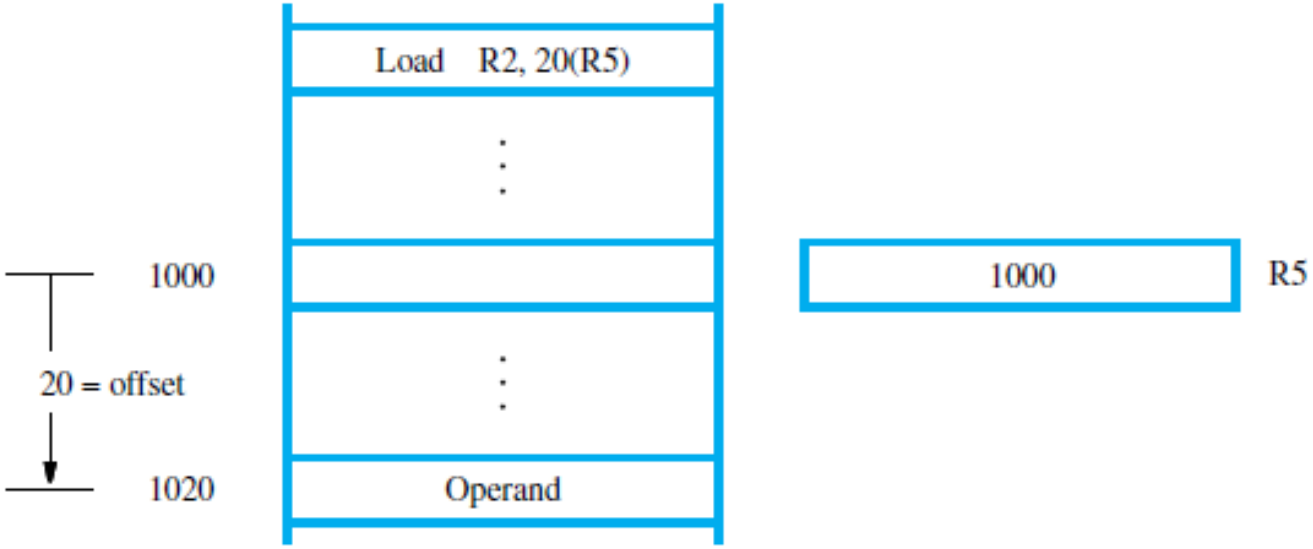
Figure 2.8 Use of indirect addressing in the program of Figure 2.6.

Indexing and Arrays



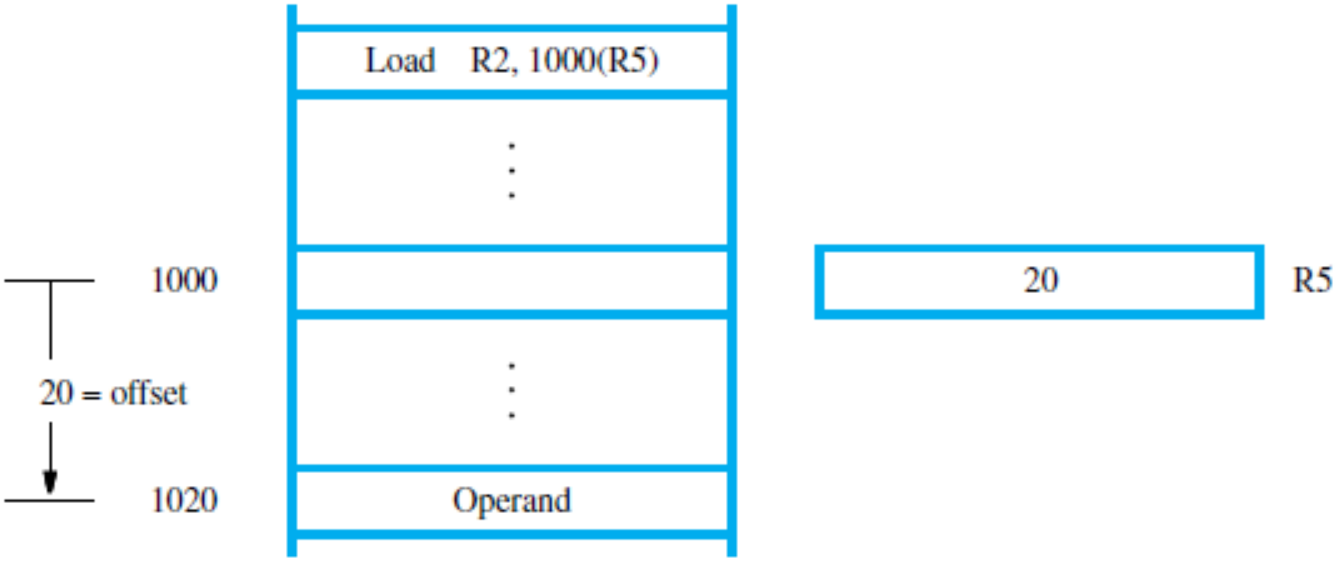
- Index mode – the effective address of the operand is generated by adding a constant value to the contents of a register.
- Index register
- $X(R_i)$; $EA = X + [R_i]$
- The constant X may be given either as an explicit number or as a symbolic name representing a numerical value.
- If X is shorter than a word, sign-extension is needed.

Indexing and Arrays



(a) Offset is given as a constant

Indexing and Arrays



(b) Offset is in the index register

Indexing and Arrays

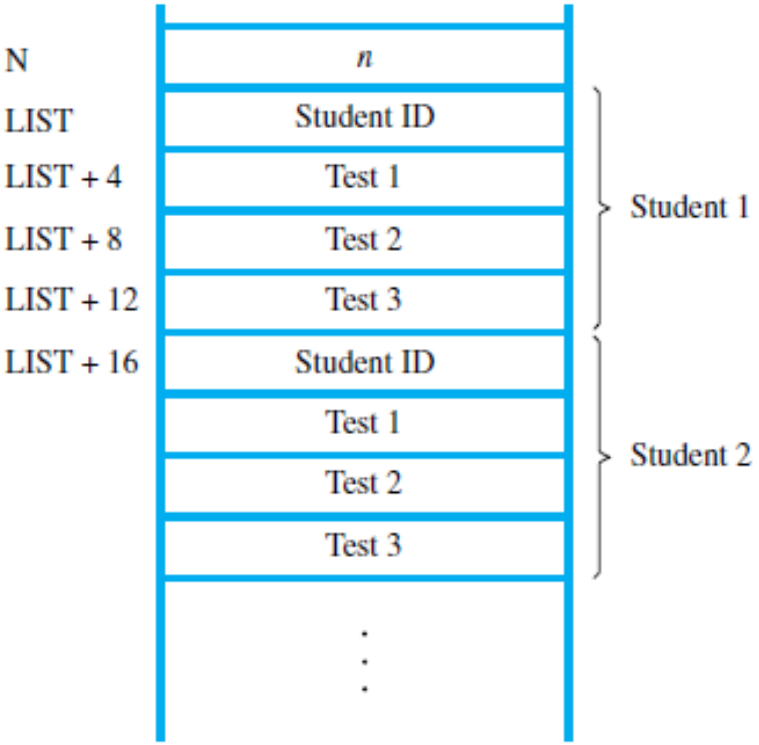
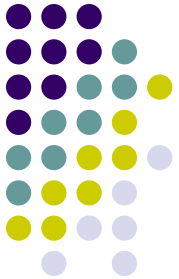


Figure 2.10 A list of students' marks.

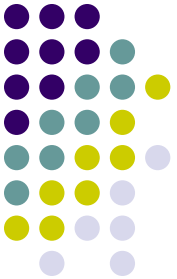
Indexing and Arrays

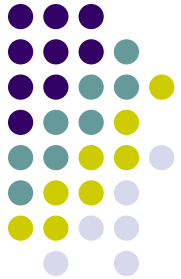


```
        Move            R2, #LIST       Get the address LIST.
        Clear           R3
        Clear           R4
        Clear           R5
        Load            R6, N           Load the value n.
LOOP:   Load            R7, 4(R2)       Add the mark for next student's
        Add             R3, R3, R7        Test 1 to the partial sum.
        Load            R7, 8(R2)       Add the mark for that student's
        Add             R4, R4, R7        Test 2 to the partial sum.
        Load            R7, 12(R2)      Add the mark for that student's
        Add             R5, R5, R7        Test 3 to the partial sum.
        Add             R2, R2, #16     Increment the pointer.
        Subtract        R6, R6, #1      Decrement the counter.
        Branch_if_[R6]>0 LOOP           Branch back if not finished.
        Store           R3, SUM1        Store the total for Test 1.
        Store           R4, SUM2        Store the total for Test 2.
        Store           R5, SUM3        Store the total for Test 3.
```

Figure 2.11 Indexed addressing used in accessing test scores in the list in Figure 2.10.

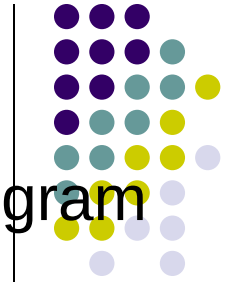
Assembly Language





- A complete set of symbolic names (mnemonics) and rules for their use constitute Assembly Language.
- Assembler program one of a collection of utility programs that are a part of the system software.
- It translates programs written in Assembly language(source pgm.) to sequence of machine instructions automatically(object pgm.).
- It may or may not be case sensitive.

Assembler Directives



- If the assembler is to produce an object program according to this arrangement, it has to know

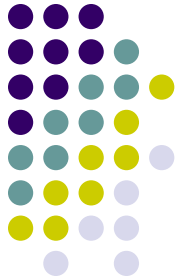
• How to interpret the names

• Where to place the instructions in the memory

• Where to place the data operands in the memory

- For this, along with instructions assembler allows the programmer to specify other information needed to translate source pgm. into object pgm. in the form of Assembler Directives.
- These are not instructions and will not appear in the object pgm.

Assembler Directives (Ref. pg. 60-61)



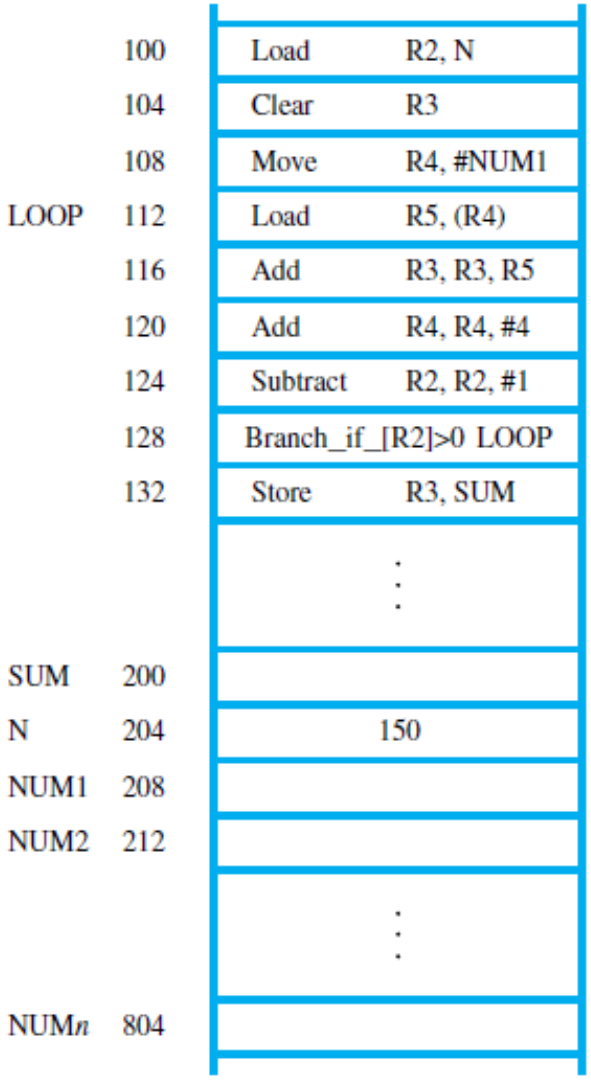
- Eg: EQU -> assigns a value to the name

  SUM EQU 200
- ORIGIN -> tells the assembler wherein memory to place the data block that follows.

  ORIGIN 100
- DATAWORD -> loads the value to variables and constants.

  N DATAWORD 100
- RESERVE -> reserves block of memory

  NUM1 RESERVE 400
- END -> indicates the end of source pgm.
- START -> address of the location at which the execution of the pgm. Is to begin.

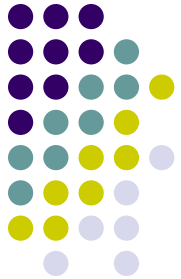


Memory address label	Operation	Addressing or data information
	ORIGIN	100
	LD	R2, N
	CLR	R3
	MOV	R4, #NUM1
LOOP:	LD	R5, (R4)
	ADD	R3, R3, R5
	ADD	R4, R4, #4
	SUB	R2, R2, #1
	BGT	R2, R0, LOOP
	ST	R3, SUM
	next instruction	
	ORIGIN	200
SUM:	RESERVE	4
N:	DATAWORD	150
NUM1:	RESERVE	600
	END	

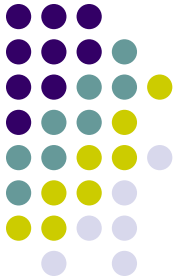
embly language representation for the program in Figure 2.12.

Figure 2.12 Memory arrangement for the program in Figure 2.8.

Assembling and Execution of Programs



- Symbol Table
- Two Pass Assembler
- Loader - Executing the loader performs a sequence of input operations needed to transfer the machine-language program from the disk into a specified place in the memory.
  - Having loaded the object code, the loader starts execution of the object program by branching to the first instruction to be executed
- Debugger - To help the user find other programming errors, the system software usually includes a *debugger* program.



Stacks



- A *stack* is a list of data elements, usually words, with the accessing restriction that elements can be added or removed at one end of the list only.
- This end is called the top of the stack, and the other end is called the bottom.
- *last-in–first-out* (LIFO) stack, pushdown stack
- The terms *push* and *pop* are used to describe placing a new item on the stack and removing the top item from the stack, respectively.
- the *stack pointer* (SP) register is used to access the stack.

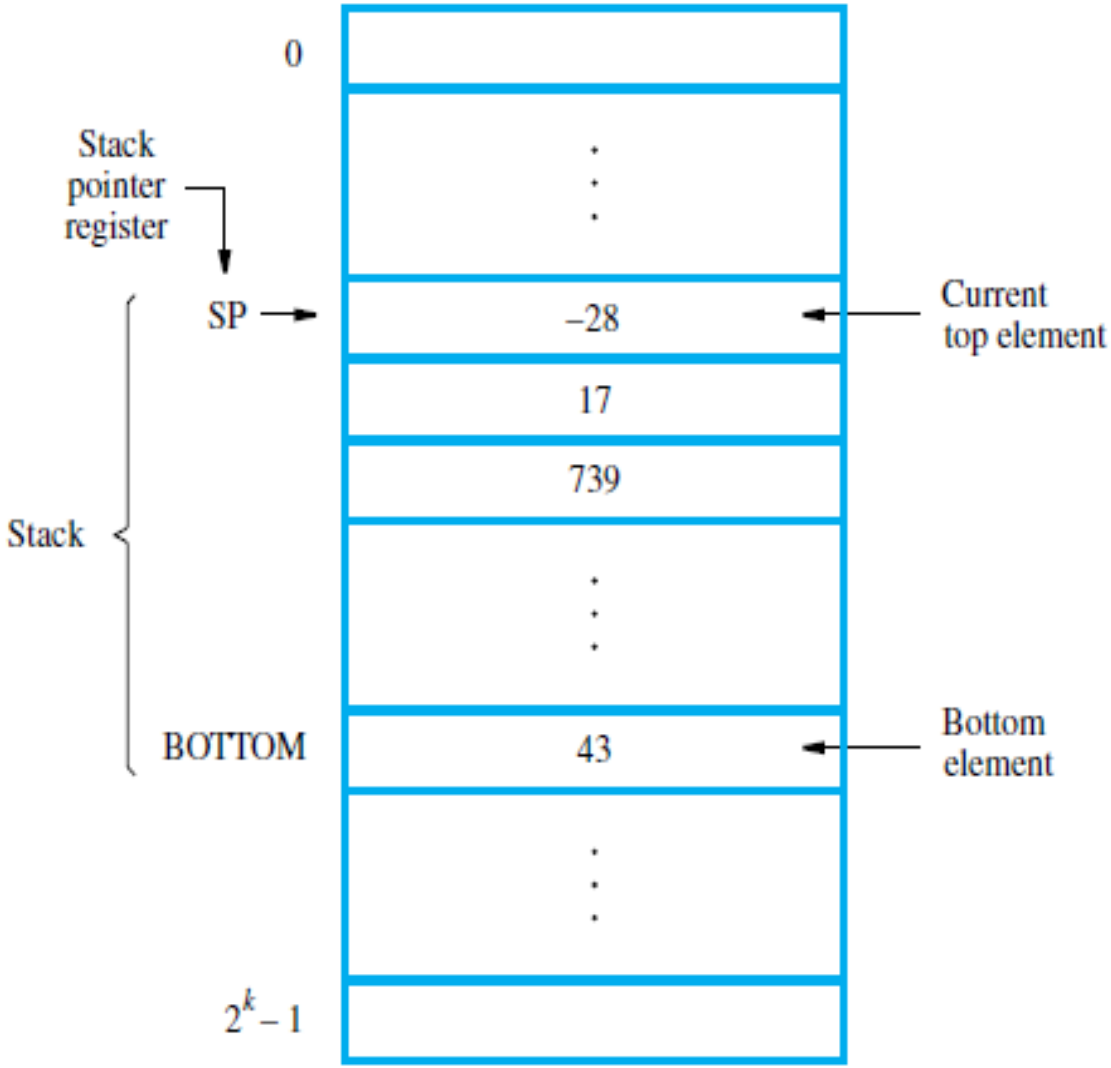


Figure 2.14 A stack of words in the memory.

- Push:

Subtract SP, SP, #4

Store R*j*, (SP)

- Pop

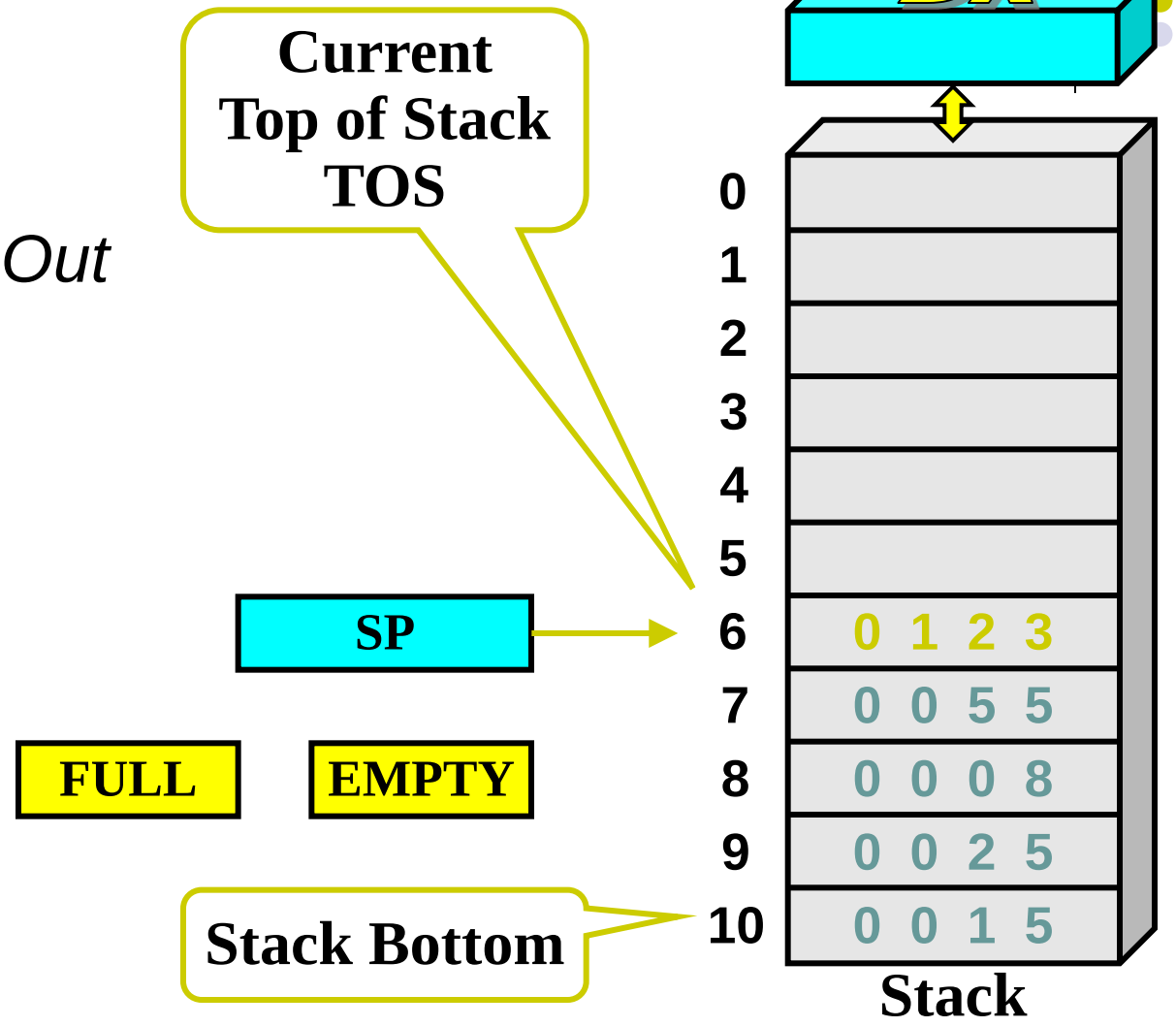
Load R*j*, (SP)

Add SP, SP, #4

Stack Organization

- LIFO

  Last In First Out



Stack Organization

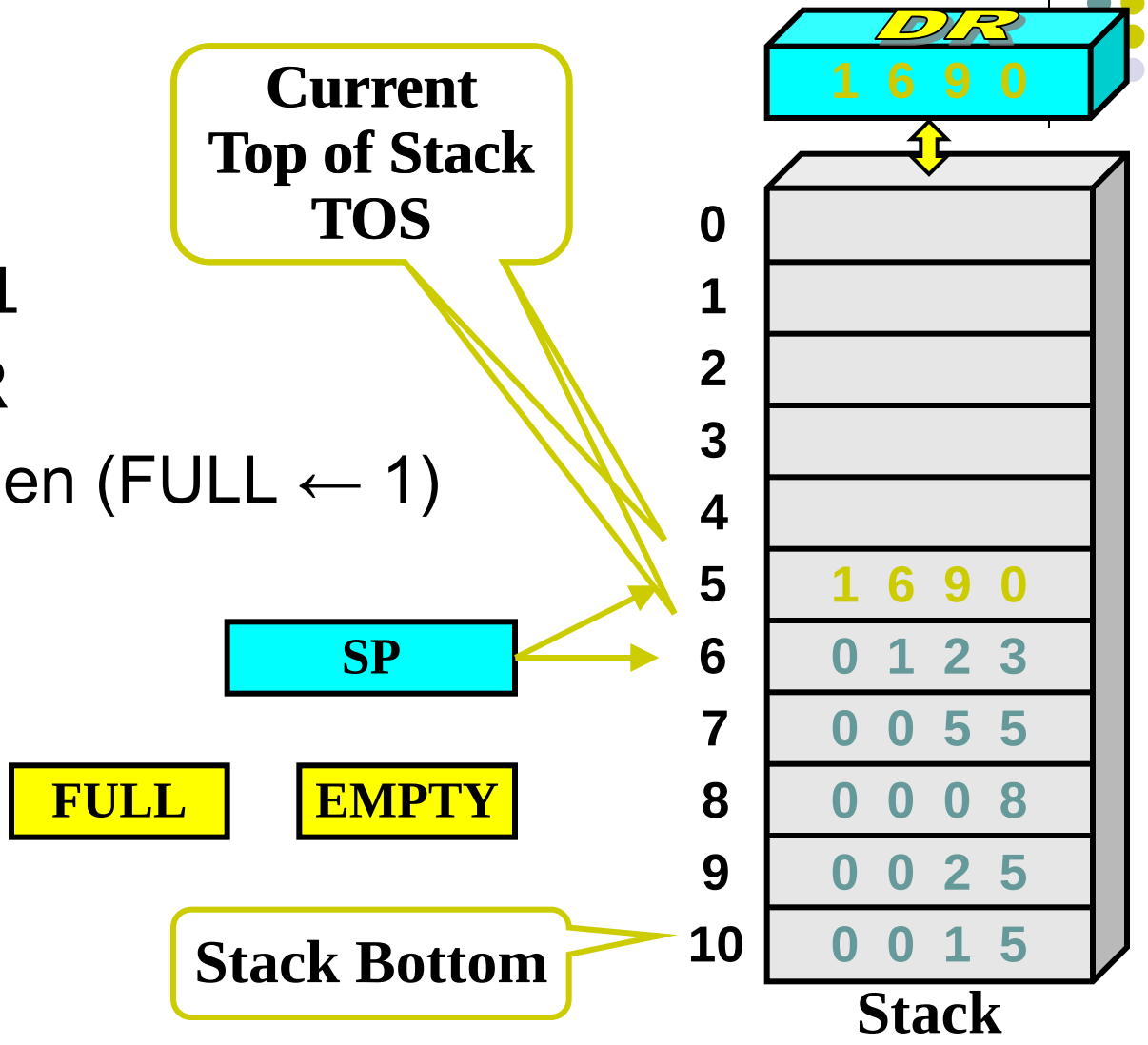
- PUSH

  SP ← SP – 1

  M[SP] ← DR

  If (SP = 0) then (FULL ← 1)

  EMPTY ← 0



Stack Organization

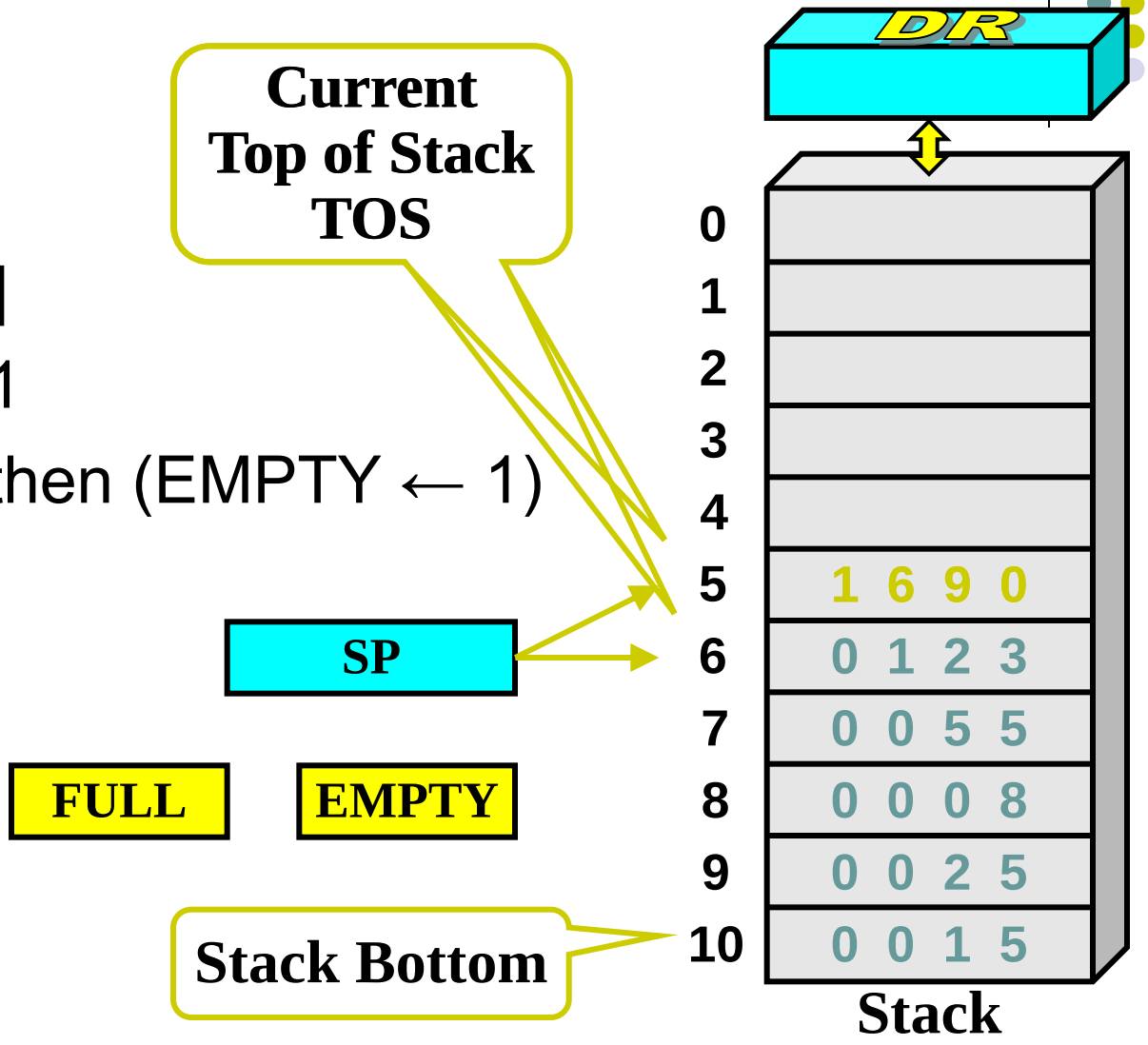
- POP

  DR ← M[SP]

  SP ← SP + 1

  If (SP = 11) then (EMPTY ← 1)

  FULL ← 0



Stack Organization

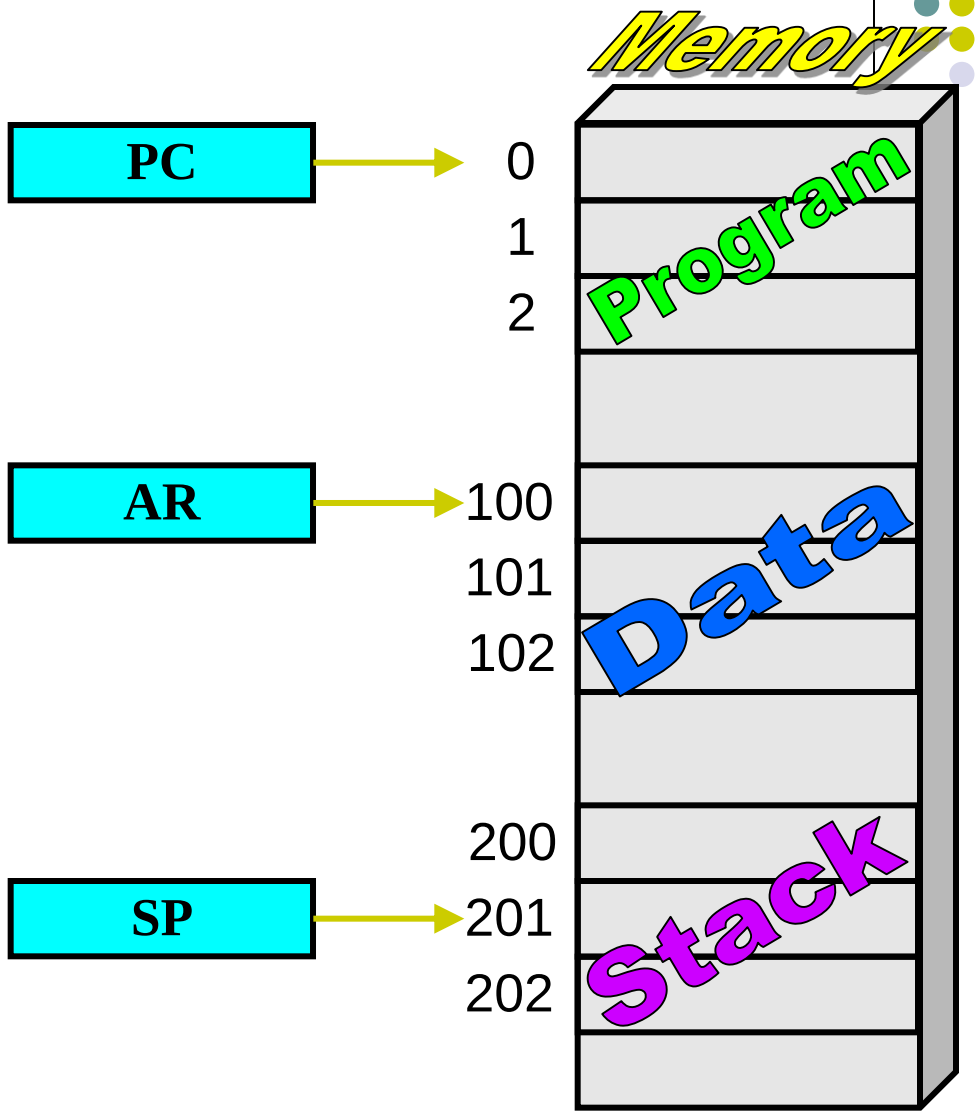
- Memory Stack
  - PUSH

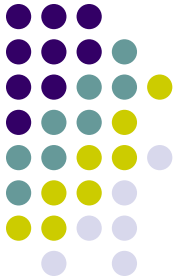
    SP ← SP – 1

    M[SP] ← DR
  - POP

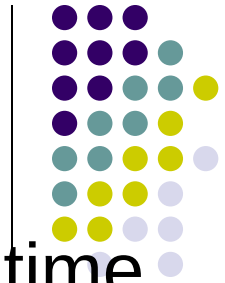
    DR ← M[SP]

    SP ← SP + 1

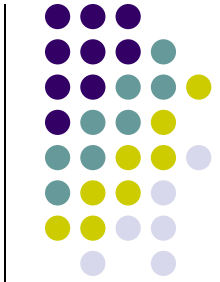




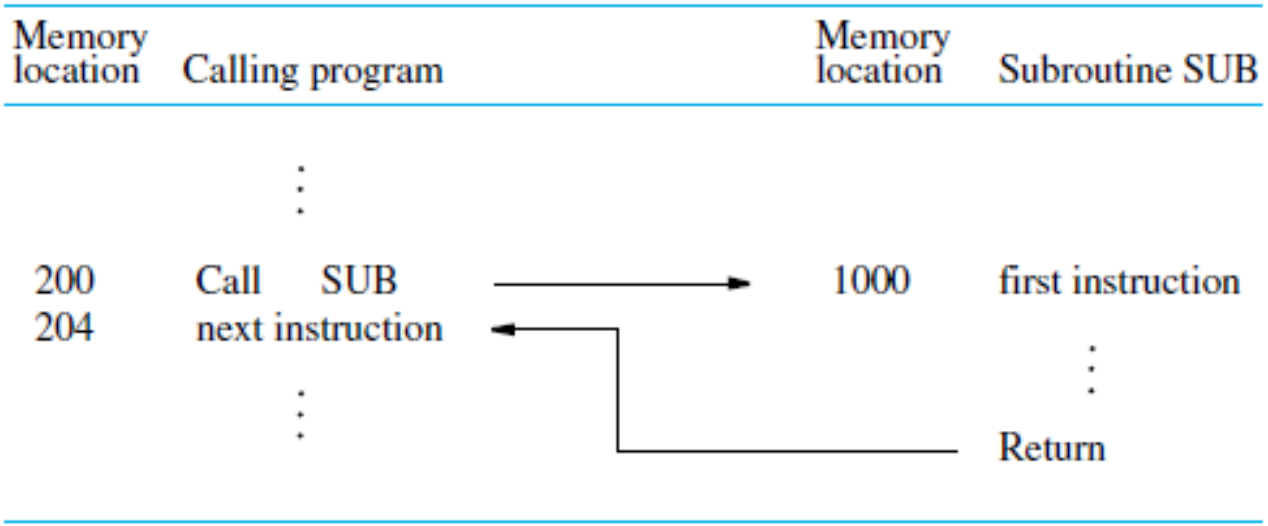
Subroutines



- A block of instructions that is executed each time a task has to be performed is called subroutine.
- Implemented using CALL and RETURN instructions.
- The way in which a computer makes it possible to call and return from subroutines is referred to as its *subroutine linkage* method.
- The simplest subroutine linkage method is to save the return address in a specific location, which may be a register dedicated to this function. Such a register is called the *link register*.



- Call instruction steps:
  - Store the contents of the PC in the link register
  - Branch to the target address specified by the Call instruction
- Return instruction steps:
  - Branch to the address contained in the link register.



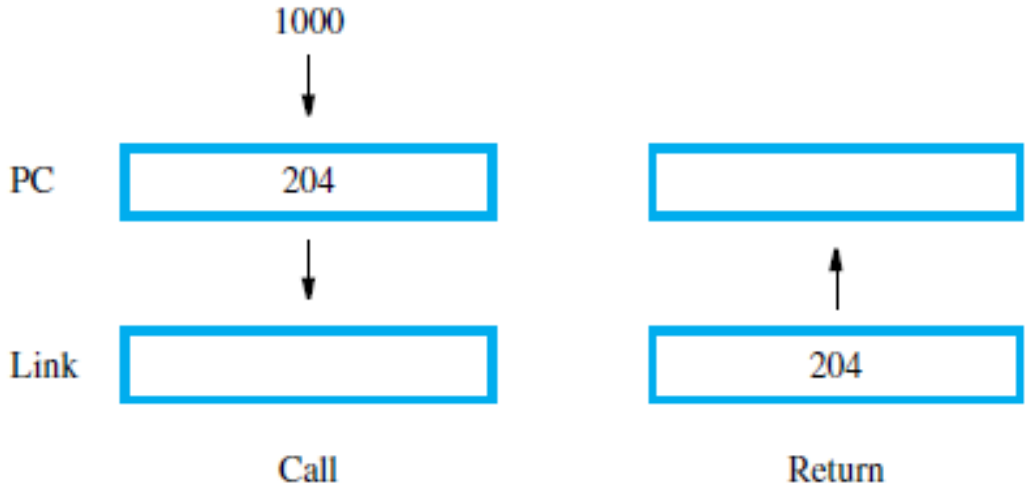
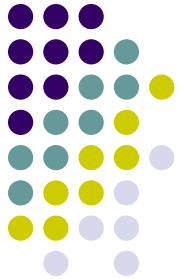


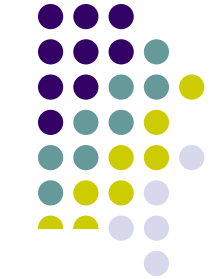
Figure 2.16 Subroutine linkage using a link register.

Subroutine Nesting and Processor Stack



- One subroutine calling another.
- Subroutine nesting can be carried out to any depth.
- The last subroutine called completes its computations and returns to the subroutine that called it.
- Return addresses associated with subroutine calls should be pushed onto the processor stack

Parameter Passing



- Registers:

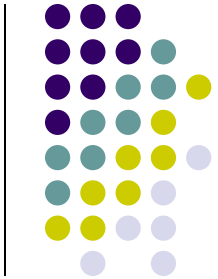
Calling program

```
                Load        R2, N          Parameter 1 is list size.
                Move        R4, #NUM1      Parameter 2 is list location.
                Call        LISTADD        Call subroutine.
                Store       R3, SUM        Save result.
                ⋮
```

Subroutine

```
LISTADD:        Subtract    SP, SP, #4     Save the contents of
                Store       R5, (SP)         R5 on the stack.
                Clear       R3             Initialize sum to 0.
LOOP:           Load        R5, (R4)       Get the next number.
                Add         R3, R3, R5     Add this number to sum.
                Add         R4, R4, #4     Increment the pointer by 4.
                Subtract    R2, R2, #1     Decrement the counter.
                Branch_if_[R2]>0 LOOP
                Load        R5, (SP)       Restore the contents of R5.
                Add         SP, SP, #4
                Return                     Return to calling program.
```

Parameter Passing

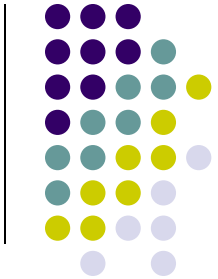


- Stack:

Assume top of stack is at level 1 in Figure 2.19.

```
Move        R2, #NUM1    Push parameters onto stack.
Subtract    SP, SP, #4
Store       R2, (SP)
Load        R2, N
Subtract    SP, SP, #4
Store       R2, (SP)
Call        LISTADD      Call subroutine
                           (top of stack is at level 2).
Load        R2, 4(SP)    Get the result from the stack
Store       R2, SUM        and save it in SUM.
Add         SP, SP, #8   Restore top of stack
                           (top of stack is at level 1).
:
```

Parameter Passing



- Stack:

```
LISTADD:  Subtract          SP, SP, #16     Save registers
          Store             R2, 12(SP)
          Store             R3, 8(SP)
          Store             R4, 4(SP)
          Store             R5, (SP)          (top of stack is at level 3).
          Load              R2, 16(SP)      Initialize counter to n.
          Load              R4, 20(SP)      Initialize pointer to the list.
          Clear             R3              Initialize sum to 0.
LOOP:     Load              R5, (R4)        Get the next number.
          Add               R3, R3, R5      Add this number to sum.
          Add               R4, R4, #4      Increment the pointer by 4.
          Subtract          R2, R2, #1      Decrement the counter.
          Branch_if_[R2]>0  LOOP
          Store             R3, 20(SP)      Put result in the stack.
          Load              R5, (SP)        Restore registers.
          Load              R4, 4(SP)
          Load              R3, 8(SP)
          Load              R2, 12(SP)
          Add               SP, SP, #16       (top of stack is at level 2).
          Return                            Return to calling program.
```

Parameter Passing

- Stack:

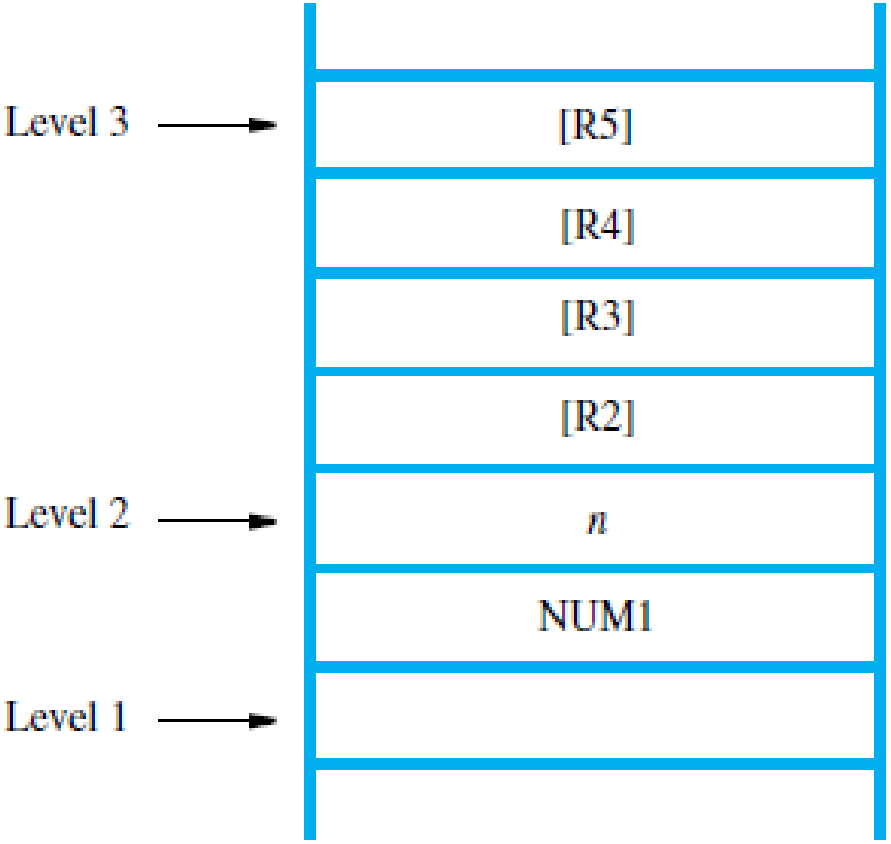


Figure 2.19 Stack contents for the program in Figure 2.18.

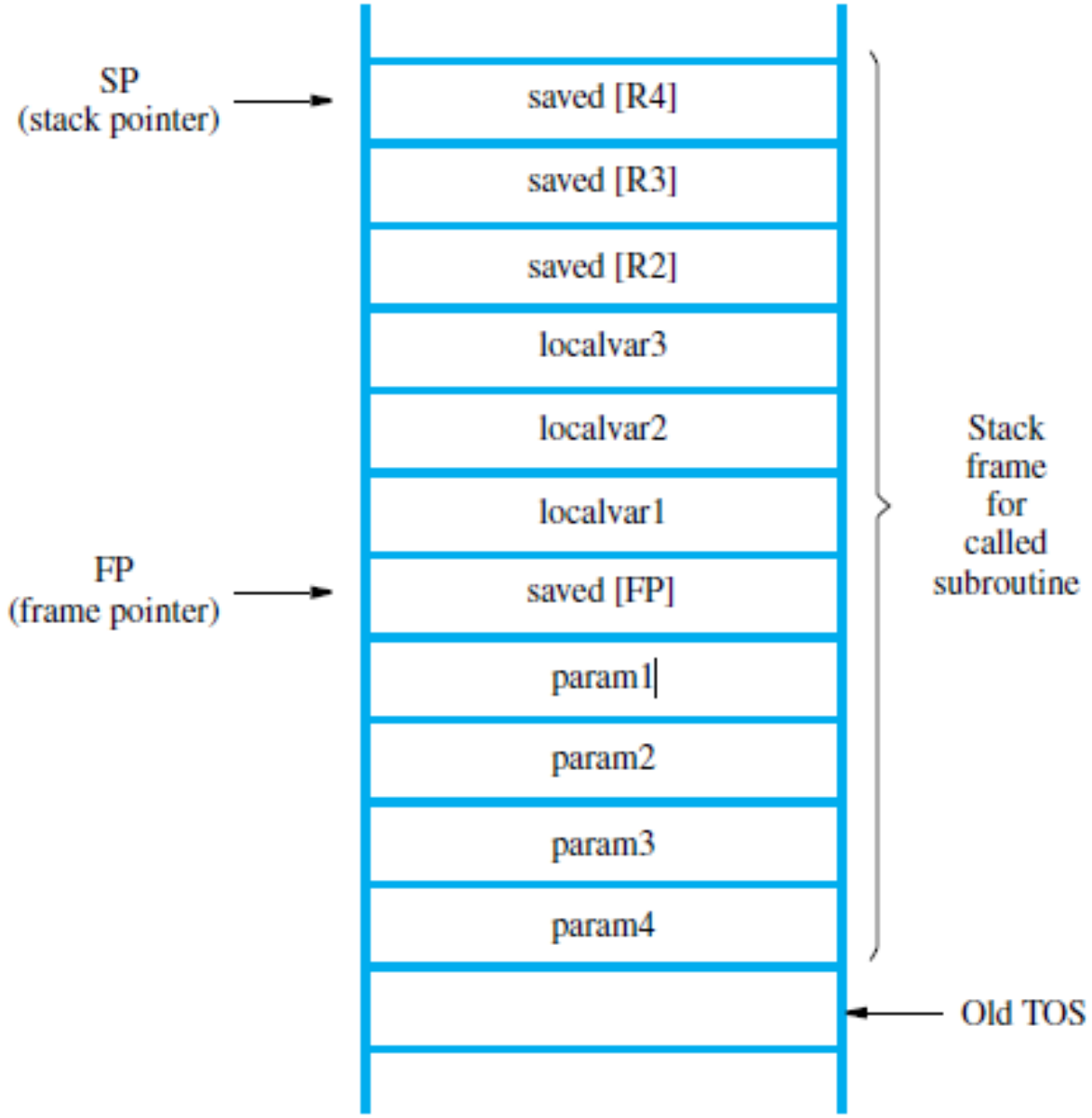
Stack Frame



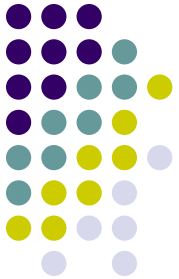
- During execution of the subroutine, locations at the top of the stack contain entries that are needed by the subroutine.
- These locations constitute a private work space for the subroutine, allocated at the time the subroutine is entered and deallocated when the subroutine returns control to the calling program. Such space is called a stack frame.
- In addition to the stack pointer SP, it is useful to have another pointer register, called the *frame pointer* (FP), for convenient access to the parameters passed to the subroutine and to the local memory variables used by the subroutine.

Stack Frame

- FP register points to the location just above the stored parameters.
- The parameters can be accessed by using addresses 4(FP), 8(FP), ...
- The local variables can be accessed by using addresses −4(FP), −8(FP), ...
- The contents of FP remain fixed throughout the execution of the subroutine, unlike SP.

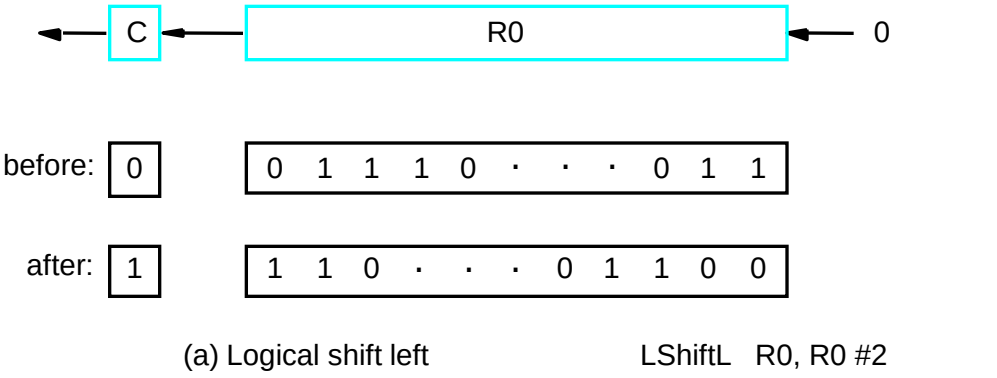


Additional Instructions



Logical Shifts

- Logical shift – shifting left (LShiftL) and shifting right (LShiftR)

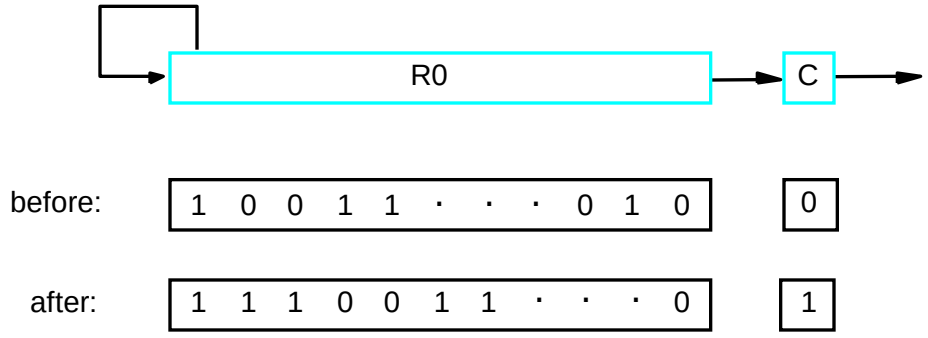


(a) Logical shift left LShiftL R0, R0 #2



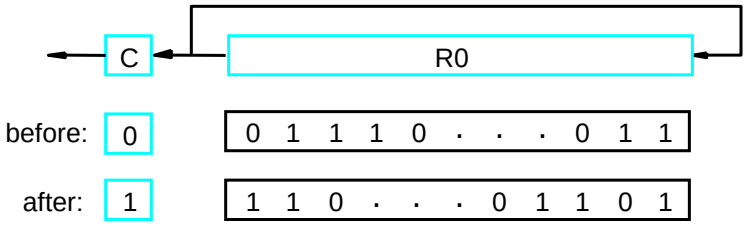
(b) Logical shift right LShiftR R0, R0 #2

Arithmetic Shifts

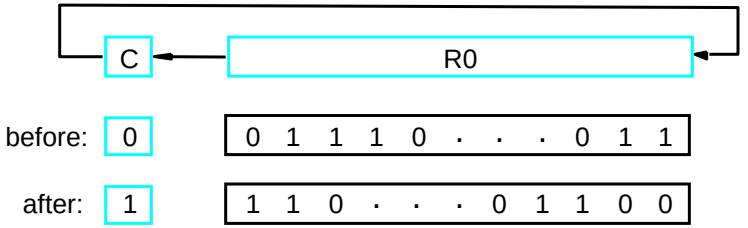


(c) Arithmetic shift right AShiftR R0, R0 #2

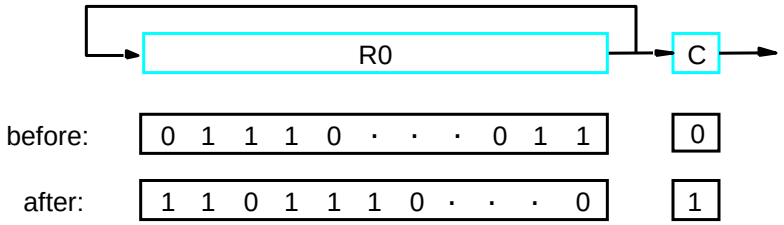
Rotate



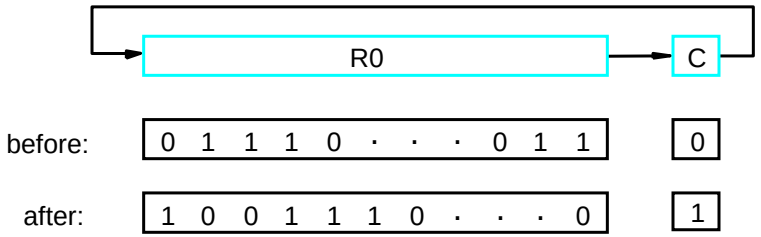
(a) Rotate left without carry RotateL R0, R0 #2



(b) Rotate left with carry RotateLC R0, R0 #2



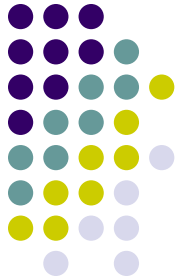
(c) Rotate right without carry RotateR R0, R0 #2



(d) Rotate right with carry RotateRC R0, R0 #2

Figure 2.32. Rotate instructions.

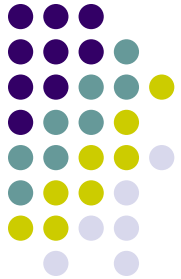
RISC VS. CISC



RISC style is characterized by:

- Simple addressing modes
- All instructions fitting in a single word
- Fewer instructions in the instruction set, as a consequence of simple addressing modes
- Arithmetic and logic operations that can be performed only on operands in processor registers
- Load/store architecture that does not allow direct transfers from one memory location to another; such transfers must take place via a processor register
- Simple instructions that are conducive to fast execution by the processing unit using techniques such as pipelining
- Programs that tend to be larger in size, because more, but simpler instructions are needed to perform complex tasks

RISC VS. CISC



CISC style is characterized by:

- More complex addressing modes
- More complex instructions, where an instruction may span multiple words
- Many instructions that implement complex tasks
- Arithmetic and logic operations that can be performed on memory operands as well as operands in processor registers
- Transfers from one memory location to another by using a single Move instruction
- Programs that tend to be smaller in size, because fewer, but more complex instructions are needed to perform complex tasks